

Final Project

Detecting Market Manipulation in High-Frequency LOB Data



Ahan Bhakare (ahaultrain)
Wissal Barday (wissalbarday)

Kaggle ID: ahaultrain & wissalbarday
FRE-GY 7773: MACHINE LEARNING

Contents

1. Introduction.....	4
1.1 Problem Statement.....	4
1.2 Dataset Overview.....	5
1.3 Evaluation.....	5
2. Data Preparation.....	5
2.1 Initial Inspection.....	5
2.2 Quality Issues Addressed.....	6
2.3 Cleaning Actions Applied.....	7
2.4 Building the Normal Pool.....	8
3. Exploratory Data Analysis.....	8
3.1 Univariate Distributions.....	8
3.2 Train vs. Validation vs. Test Comparison.....	10
3.3 Per-Symbol Heterogeneity.....	12
3.4 Joint Feature Structure.....	13
3.5 Manipulation Pattern Fingerprints.....	15
3.6 EDA Synthesis.....	18
4. Feature Engineering.....	19
4.1 Burst Features.....	19
4.2 Per-Symbol Z-Scores.....	20
4.3 Magnitude and Spread Features.....	21
4.4 The 37-Feature Set.....	22
4.5 Scaling.....	22
5. Simple Supervised and Unsupervised Models.....	23
5.1 The Semi-Supervised Challenge.....	23
5.2 Synthetic anomaly injection.....	24
5.3 Model 1: Logistic Regression.....	25
5.4 Model 2: Random Forest.....	27
5.5 Model 3: Linear SVM.....	28
5.6 Model 4: Dense Autoencoder.....	30
5.7 Initial Kaggle results.....	34
5.8 First ensemble (RF + AE).....	35
6. Advanced Anomaly Detectors.....	36
6.1 Isolation Forest.....	36

6.2 Z-score detector	38
6.3 Two-model ensemble (IF + AE)	41
6.4 Final 3-way ensemble (Z-score + IF + AE)	42
7. Results and Discussion	44
7.1 Leaderboard progression.....	44
7.2 What worked and why	44
7.3 Limitations	46
8. Conclusion	47
References.....	48

1. Introduction

1.1 Problem Statement

Financial markets generate a huge number of order-book events every second. Most of these events signal normal trading activity, but some are designed to manipulate prices, create false signals, or mislead other market participants. In this project, we focus on detecting suspicious behavior in high-frequency limit order book data, with particular attention to three main types of manipulation:

- Spoofing/layering
- Pump-and-dump activity
- Quote stuffing

These three manipulation types are different, so they do not necessarily leave the same footprint within the data. Spoofing and layering is usually linked to fake liquidity: a trader places large orders to create the impression of buying or selling pressure, then cancels those orders before execution. In the order book, this can appear through unusual cancellation patterns, large cancelled sizes, or an imbalance between trades and cancellations. Pump-and-dump behavior is more connected to abnormal price movement, where returns may show sudden jumps or reversals that do not look like normal market activity. Quote stuffing, on the other hand, is linked to very rapid sequences of orders and cancellations, where the suspicious signal comes less from one individual row and more from burst-like order activity.

The main challenge is that we are not given labelled fraud examples in the training data. The train and validation sets only contain normal observations, while the test set contains a hidden mix of normal and anomalous rows. Instead of learning directly from examples of fraud, we need to learn what “normal” market behavior looks like and then flag rows that strongly deviate from it.

Because these manipulation strategies show up in different ways, the detection approach cannot depend on one signal alone. Some suspicious events may appear through abnormal return movements, while others may show up through unusually large cancellations, strange trade-to-cancellation relationships, or repeated burst patterns in the order sequence. This is why the project combines several types of features instead of relying only on raw price movements.

The broader motivation is that market manipulation directly affects fairness and trust in financial markets. When these strategies go undetected, they can distort prices and give unfair advantages to traders who are intentionally exploiting the order book. For this reason, the project is built around two goals: achieving strong predictive performance and keeping the detection process interpretable enough to understand which signals are actually driving the results.

1.2 Dataset Overview

The dataset comes from a Kaggle competition for the 7773 course and contains limit order book snapshots for five NASDAQ stocks: AAPL, AMZN, GOOG, INTC, and MSFT. Each row represents one timestamped market event and includes 14 numerical features related to price returns, order-book depth, executed trades, cancellations, and activity indicators.

The training and validation sets are fully normal, while the test set contains the hidden anomalies used for scoring. Because of this setup, we combined the train and validation data into one normal pool and used it to estimate normal market behavior. The test set was never used when fitting scalers, models, or training statistics.

1.3 Evaluation

The competition evaluates submissions using the Area Under the ROC Curve, or **AUROC**. AUROC measures how well the model ranks anomalous observations above normal observations. An AUROC of 0.50 means the model is no better than random guessing, while a score close to 1.00 means the model separates normal and anomalous rows very well. The competition baseline is 0.88, and our final submitted model achieved a public leaderboard AUROC of 0.99473.

2. Data Preparation

Before building any model, we started by looking closely at the three datasets to see what needed to be cleaned or adjusted. Since the project used separate train, validation, and test files, even small differences in column names, data types, or missing values could create problems later in the modelling pipeline. The goal of this section is therefore to inspect the structure of each set, identify inconsistencies, and clean the data without leaking data from the test set into the training process.

2.1 Initial Inspection

We began by loading the three CSV files and comparing their basic properties: number of rows, column names, data types, and missing values (as depicted in Figure 1). This first inspection showed that the train and validation files were very similar. They had the same columns, the same data types, and the same target structure. This made sense because both files represent normal market behavior and both contain *TARGET* = 0.

Rows	1,070,022	467,939	289,101
Columns	23 (incl. ClusterNo)	23 (incl. ClusterNo)	21
TARGET column	Yes, all 0 (normal)	Yes, all 0 (normal)	Not provided
ClusterNo column	Present, 100% NaN	Present, 100% NaN	Absent
OriginalSequenceNumber dtype	int64	int64	object (string)
Null values in features	0	0	278
Symbols	5 (AAPL, AMZN, GOOG, INTC, MSFT)	5 (balanced)	5 (dominated by INTC + MSFT)

Figure 1: Structural Comparison of the 3 Data Sets

The test file, however, had a slightly different structure. First, it did not contain the *TARGET* column, which is expected because the test labels are hidden for the Kaggle competition. Second, it did not contain the *ClusterNo* column. Third, the row identifier was named *index* instead of *ID*, which would cause issues when creating the final submission file. Finally, the *OriginalSequenceNumber* column had a different format in the test set: instead of appearing as plain integers, some values appeared as dashed strings such as "16120456-000001-026".

This difference in *OriginalSequenceNumber* was especially important. At first, it looked like a formatting inconsistency, but it actually contained useful information. The dashed format indicates that multiple events may belong to the same base order sequence. For example, "16120456-000001-026" can be interpreted as a sub-event linked to the base sequence 16120456. This later became useful for building burst-order features, which helped capture quote-stuffing behavior. For this reason, we did not simply delete or force-convert this column during the initial cleaning step.

2.2 Quality Issues Addressed

The initial inspection revealed four main data quality issues that needed to be addressed before feature engineering and modelling:

- *ClusterNo* was 100% missing in the train and validation sets, and it was absent from the test set. This meant that the column carried no usable information and also created an inconsistency between the available features in the training files and the test file.
- The test row identifier was named *index* instead of *ID*. This issue appears only in the test set. If left unchanged, it could create problems when preparing the final Kaggle submission, because each anomaly score needs to be matched to the correct test row using a consistent ID column.
- *OriginalSequenceNumber* had different formats across the splits (seen in Figure 2). In train and validation, the column appears as a plain numeric sequence, while in the test set it sometimes appears as a dashed string. This mattered because the column could not simply be treated as the same numeric variable across all files.

- 278 missing values appeared in the test size and indicator columns. These values needed to be handled before modelling, because missing inputs could cause scalers or models to fail, or produce invalid anomaly scores.

train	48222841, 48305744, 48306207	int64	Plain sequence numbers
val	54446715, 54446720, 54446721	int64	Plain sequence numbers
test	'16113575', '16120456', '16120456-000001-000'	object	Mix: plain integers and dashed sub-event strings

Figure 2: *OriginalSequenceNumber* Format Examples by Split

The most important point is that these issues were mostly structural, not evidence of poor-quality data. The train and validation sets were already clean in terms of labels and feature availability. The main work was making sure that the test file could be processed through the same pipeline without breaking the feature engineering code.

2.3 *Cleaning Actions Applied*

After identifying these issues, we applied a small number of cleaning steps to make the three files consistent while preserving useful information for later feature engineering.

The first cleaning action was to drop *ClusterNo* from the train and validation sets. Since the column was entirely missing and did not appear in the test file, it could not contribute useful information to the model. Keeping it would only add an empty feature and make the datasets less consistent.

The second action was to rename the test identifier column from *index* to *ID*. This was necessary because the final submission file needed to match each anomaly score to the correct test row. By giving the test file the same identifier name as the train and validation files, we made the later submission step simpler, less error-prone, and more consistent.

The third decision involved *OriginalSequenceNumber*. We did not remove or force-convert this column during initial cleaning because the dashed strings in the test set turned out to contain meaningful order-sequence information. Instead, we kept the column and handled it later during the feature engineering step. In that step, we extracted the base sequence prefix and counted how many rows shared the same base order. This became the basis for the burst features used to detect quote-stuffing patterns.

The fourth cleaning action was to handle the 278 missing values in the test set. These values appeared in size and indicator columns. During feature engineering, we filled them with 0. This choice was consistent with the structure found later in the EDA: several trade, cancellation, and indicator columns had repeated values representing a “no event” state. In this context, filling missing size or indicator values with 0 was a reasonable way to represent the absence of recorded activity without creating artificial extreme values.

2.4 Building the Normal Pool

After cleaning the structural issues, we combined the train and validation sets into one normal dataset. This gave us a normal pool of 1,537,961 rows, all with *TARGET* = 0. Since both files only contain normal observations, keeping them separate would have reduced the amount of data available to estimate normal market behavior. By combining them, we created a larger and more stable reference distribution for the rest of the pipeline.

This normal pool was used to fit all training-based components, including feature statistics, means, standard deviations, scalers, and unsupervised anomaly detection models. The test set was never used when fitting these components, because doing so would introduce data leakage and make the evaluation less reliable.

The role of the normal pool was therefore to define what “normal” order-book behavior looks like. Once these parameters were fitted, the test set was transformed using the same rules, and each test row was scored based on how strongly it deviated from the normal reference. This setup keeps the pipeline consistent with the competition structure: we learn only from confirmed normal data and then rank the hidden test rows by anomaly score.

3. Exploratory Data Analysis

3.1 Univariate Distributions

We started the EDA by looking at each feature one column at a time, before doing any comparison across splits or symbols. The goal here was to understand what a typical value looks like in every column and to spot anything in the distribution that could affect modelling later. Because the dataset arrives already pre-processed in some standardized form, this step was less about reading raw price levels and more about checking the scale, shape, and central tendency of each numeric column.

The descriptive statistics confirmed that every numeric column has been heavily rescaled before we received it. For each feature in the training set, the mean sits very close to zero and the standard deviation is close to one. *ReturnBid1* has a mean of 0.002 and a standard deviation of 0.92, *BidSize1* has a mean of 0.04 and a standard deviation of 0.99, and the other columns roughly follow the same pattern. Validation behaves almost identically. What stood out, though, was the range. Even though the typical row sits within a unit interval, the minimum and maximum values reach far into the tails: *ReturnBid1* spans -25.8 to +25.8, *DerivativeReturnBid1* spans -36.2 to +36.1, and *AskSize1* reaches 52.4 on the positive side. The bulk of the data is centered and unit-scaled, but the tails are heavy (as seen in Figure 3). This was the first sign that anomaly detection on this dataset would need to deal with extreme but rare values, and it is part of what eventually pushed us toward RobustScaler instead of StandardScaler.

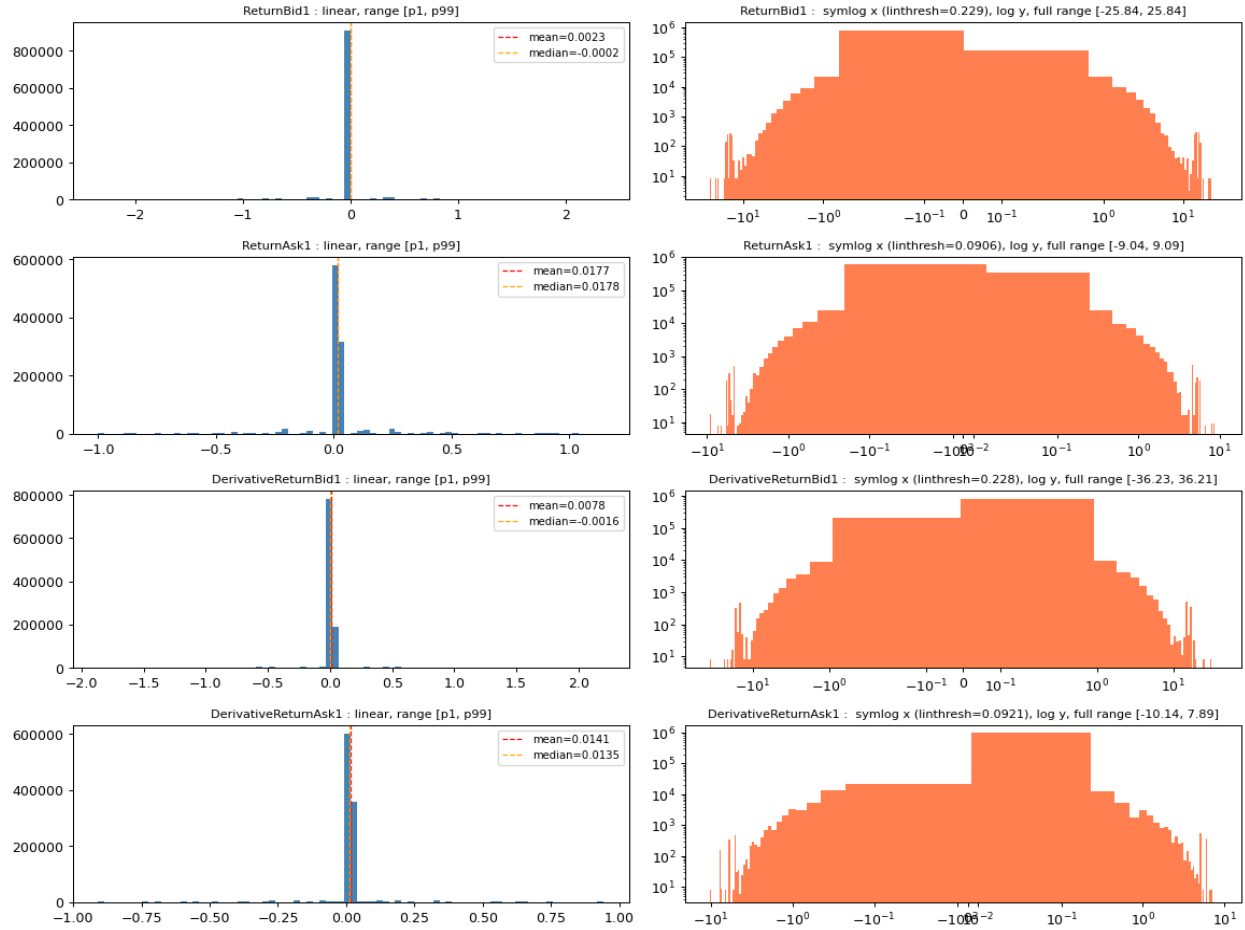


Figure 3: Z-score Centered Distribution with Heavy Tails

The categorical and temporal summaries revealed a structural asymmetry across splits that became important later. All three splits contain the same five NASDAQ symbols (AAPL, AMZN, GOOG, INTC, MSFT). Train and validation are roughly balanced across these symbols, each one holding 17 to 23 percent of rows. The test set, on the other hand, is heavily concentrated on INTC and MSFT, which together account for close to 70 percent of all test rows. The *Date* column tells a similar story: train and validation cover many distinct day markers spread over multiple years, while every row in the test set falls on a single trading day. This means that any train-versus-test comparison is partly contaminated by date and symbol differences before we even look at anomalies, which we revisit in Section 3.2.

Looking at the most common values inside each numeric column then revealed a pattern that the descriptive statistics could not show. Several columns are dominated by a single repeated value that covers roughly twenty percent of all rows. *TradeBidSize* and *TradeAskSize* each have about twenty percent of their training rows piled on one negative value, and the matching *TradeBidIndicator* and *TradeAskIndicator* columns show the same concentration. The return

and derivative-return columns have an even stronger spike, with their top-1 mode covering 21 percent of rows. These mass points correspond to the standardised "no event" state for each column: when no actual trade or cancellation occurred for a row, the column takes one fixed standardised value rather than a true zero. This matters because a feature that looks well-spread in the descriptive statistics can still be dominated by a single mass point. Any scaler that relies on the mean and standard deviation would be pulled toward this repeated value, and any anomaly score that ignores it could end up modelling the no-event state instead of the actual variability we care about.

Finally, we plotted each numeric feature in two views side by side (Figure 3): a linear histogram zoomed to the [p1, p99] range, and a symlog-scale histogram covering the full range of the column. The linear view shows where the bulk of the distribution sits and confirms the near-zero centring already observed in the summary statistics. The symlog view shows what the linear plot hides, which is the heavy positive tail that extends well beyond the typical observation. For the returns and derivatives, the linear histogram shows a tight pile near zero, while the symlog version makes it clear that values reaching plus or minus thirty are present, just rare. For the indicator columns, the symlog view also makes the no-event mass-spike visible at one or two negative values, while the positive tail captures the rarer trading and cancellation events. Both views are needed because they answer different questions: the linear view describes a typical row, while the symlog view describes the rare but informative outliers that anomaly detection will need to handle.

3.2 Train vs. Validation vs. Test Comparison

After understanding each split on its own, we compared the three splits to each other to check whether they behave consistently. This step matters because the train and validation sets are both 100% normal, so they should look almost identical. If they don't, that would point to a sampling issue, leakage, or some temporal shift between the two sets. Test, on the other hand, is expected to differ from train and validation because it contains the hidden anomalies. The harder question is whether those differences come from the anomalies themselves or from structural confounds such as the symbol mix and the single-day date coverage that we already flagged in Section 3.1.

feature		
TradeBidSize	0.0134	0.6463
DerivativeReturnAsk1	0.0125	0.4580
TradeBidIndicator	0.0135	0.4512
ReturnBid1	0.0152	0.4490
TradeAskSize	0.0140	0.4447
ReturnAsk1	0.0159	0.4414
CancelledBidIndicator	0.0136	0.3748
TradeAskIndicator	0.0195	0.3283
DerivativeReturnBid1	0.0114	0.2920
CancelledAskIndicator	0.0132	0.2599
BidSize1	0.0350	0.1501
CancelledAskSize	0.0159	0.1411
AskSize1	0.0369	0.1375
CancelledBidSize	0.0338	0.1286

Figure 4: KS Test Data Similarity Comparison

The first piece of evidence came from a side-by-side summary table. For each numeric feature, we listed the mean, standard deviation, and median across all three splits, and added delta columns showing how far each statistic in validation and test drifts from the corresponding statistic in train. Sorting the table by the absolute mean delta against the test set made the worst offenders rise to the top. Train and validation moved together: the difference in mean is tiny for every column, with absolute deltas under 0.10 across the board, and the standard deviations tracking each other closely in most columns. The test, however, looks visibly different. *TradeAskSize* shifts from a train mean of 0.005 to a test mean of 0.29, and its standard deviation jumps from 0.98 in train to 7.04 in test. The return columns show a similar pattern, with *ReturnBid1*'s standard deviation widening from 0.92 in train to 17.98 in test, even though its mean barely moves. The pattern is consistent: the test set does not shift the centre of the distributions by much, but it does inflate the tails substantially. That is what we would expect if the test set contains rare but extreme anomalous events on top of the same underlying normal behaviour.

To put a number on these differences rather than rely on the visual impression, we ran two-sample Kolmogorov-Smirnov tests on every numeric feature, once for train versus validation and once for train versus test (Figure 4). The KS statistic D measures the largest gap between the two empirical cumulative distributions and lies between zero and one, where zero means the two distributions are indistinguishable and values close to one mean they are very different. We focus on D rather than the p-value because our sample sizes are very large (1.07M, 468K, and 289K rows). With samples of this size, any non-trivial difference produces a p-value indistinguishable from zero, so the p-value carries essentially no information for ranking features. To read D in a consistent way, we use a simple grid: under 0.02 means the two distributions are almost identical, 0.02 to 0.10 is a small difference, 0.10 to 0.20 is moderate, and anything above 0.20 counts as substantial drift.

The results show two very different patterns. For train versus validation, every D statistic sits at or below 0.04, with most features under 0.02. By the grid we defined above, that puts train and validation firmly inside the "almost identical" band. The validation set therefore behaves like a healthy second sample of normal rows, and we can use it without worrying about hidden splits or leakage. For train versus test, the picture is reversed. *TradeBidSize* is the worst feature by a wide margin, with $D = 0.6463$, well into the substantial-drift band. Five other features (*DerivativeReturnAsk1*, *TradeBidIndicator*, *ReturnBid1*, *TradeAskSize*, *ReturnAsk1*) sit above $D = 0.40$, and only four (*BidSize1*, *CancelledAskSize*, *AskSize1*, *CancelledBidSize*) stay below the 0.20 substantial-drift threshold against test. The overall pattern is that the trade-related and return-related columns drift the most, while the bid- and ask-side size columns drift the least.

The main consequence for modelling is that the train-versus-test gap is real, but it is also partly contaminated. Some of the drift is a genuine anomaly signal: the wide standard deviations and the high D values on the return and trade-size columns line up with the kind of extreme behaviour we associate with manipulation. But part of the drift is also structural, since the test set covers only one trading day and is dominated by INTC and MSFT, while the train set spans many more days and is balanced across all five symbols. We could not fully disentangle these two sources of drift

from this analysis alone, which is why later sections fit unsupervised models on the normal pool and score test rows by deviation rather than by direct distributional comparison.

3.3 Per-Symbol Heterogeneity

The previous section already hinted that *BidSize1* and *AskSize1* behaved differently across splits. To understand whether that came from anomalies, from the test set's different symbol mix, or from something more fundamental about how each stock trades, we looked next at how the five symbols compare to each other within the training data. Each stock has its own price level, typical bid-ask depth, and trading rhythm, so what counts as a normal value for AAPL might be unusual for GOOG and vice versa. If most features are roughly invariant across symbols, we can fit a single global model. If they are not, the model risks confusing genuinely unusual rows with rows that simply belong to a less-represented symbol.

	train	train_pct	val	val_pct	test	test_pct
ExternalSymbol						
AAPL	190575	17.8100	83980	17.9500	35176	12.1700
AMZN	212912	19.9000	90760	19.4000	29807	10.3100
GOOG	215865	20.1700	93212	19.9200	23521	8.1400
INTC	226109	21.1300	94578	20.2100	103496	35.8000
MSFT	224561	20.9900	105409	22.5300	97101	33.5900

Figure 5: Rows per Ticker

We started by counting how many rows each symbol contributes in each split (Figure 5). Train and validation are roughly balanced: every symbol holds between about 17 and 23 percent of rows, with AAPL the smallest in both files (17.81% in train, 17.95% in val) and INTC or MSFT the largest. The test set is the opposite. INTC accounts for 35.80% of rows and MSFT for 33.59%, so together they cover 69.4% of the test set. AAPL drops to 12.17%, AMZN to 10.31%, and GOOG to just 8.14%. This means the test split is essentially a two-symbol problem with three smaller minority groups, and any train-versus-test comparison from Section 3.2 is partly comparing one symbol mix to a very different one.

To check whether the symbols actually behave differently feature by feature, we computed the mean and standard deviation of each numeric feature separately for each symbol, using only the training set so that no test information leaked in. For the means, we added a spread column equal to the maximum minus the minimum across the five symbols, which tells us how much the centre of each feature varies between stocks. For the standard deviations, we added a ratio column equal to the maximum divided by the minimum across symbols, which tells us how much the scale of each feature varies.

The means turned out to be fairly close across symbols. The largest between-symbol spread is *BidSize1* with 0.164, followed by *CancelledBidSize* at 0.119 and *CancelledAskSize* at 0.089. Every other feature has a spread under 0.07. Even the largest of these gaps is small in absolute terms because the data is on a near-unit scale, so the centre of each feature is reasonably symbol-invariant. The standard deviations told a different story. *BidSize1* stands out sharply, with a max/min ratio

of 3.15 across symbols. AAPL has a *BidSize1* standard deviation of 0.38, while GOOG sits at 1.19, AMZN at 1.07, INTC at 1.05, and MSFT at 0.99. So for AAPL, a deviation of one unit in *BidSize1* is genuinely unusual, while for GOOG the same value is well within typical behaviour. Every other feature has a between-symbol std ratio under 1.7, with *CancelledBidSize* at 1.69, *CancelledAskSize* at 1.55, *ReturnAsk1* at 1.49, and *ReturnBid1* at 1.43. *BidSize1*'s ratio is almost twice as high as the next-most-heterogeneous feature.

The main takeaway is that the central tendency of the data is reasonably consistent across symbols, but the scale is not, and the gap is concentrated in one feature in particular. *BidSize1* is the standout heterogeneous column: it has the largest mean spread and a standard deviation ratio that dwarfs every other feature. This is what eventually motivated the per-symbol z-score features in Section 4.2. By computing $|x - \mu_{\text{sym}}| / \sigma_{\text{sym}}$ separately for each stock, we avoid flagging GOOG's normal *BidSize1* variability as anomalous just because the same value would be extreme for AAPL. A simpler global standardisation would have masked this difference and pulled the anomaly scores toward whichever symbols dominate the data, which in test means INTC and MSFT.

3.4 Joint Feature Structure

So far we have looked at every feature one column at a time, comparing distributions across splits in Section 3.2 and across symbols in Section 3.3. But a row in a limit-order-book stream is not just 14 independent measurements. Anomalies can also hide in feature combinations: a return that is normal in isolation but paired with a trade size that does not match, or a cancellation indicator that fires without any matching trade activity. To check whether the joint structure of the data carries information that the univariate analysis missed, we ran two more checks. The first was a pair of correlation matrices to see whether features fall into related blocks (Figure 6), and the second was a Mahalanobis distance distribution to see how well a simple multivariate-Gaussian model could describe normal behaviour (Figure 7).

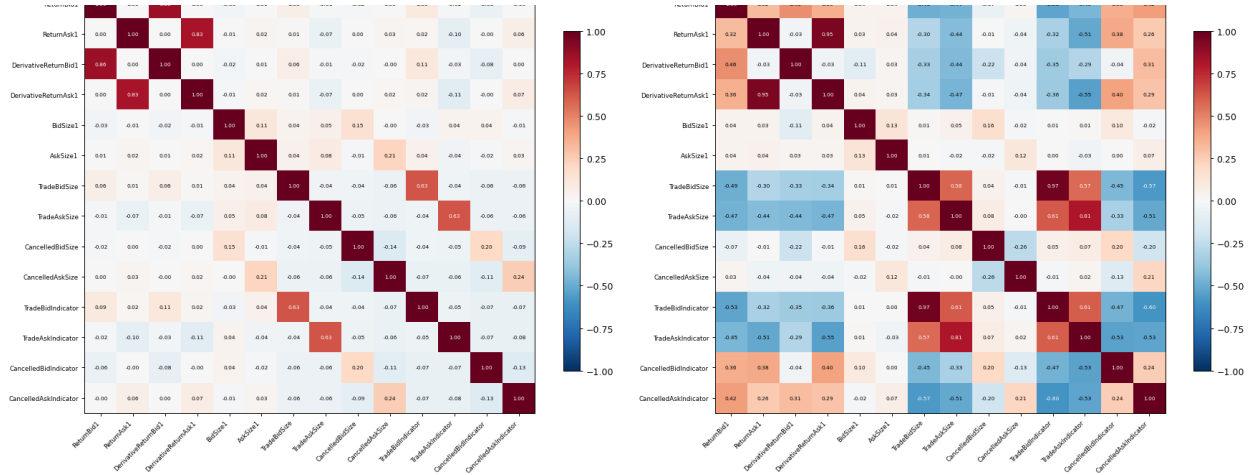


Figure 6: Pearson and Spearman Matrices

We started with the correlation matrices. To keep the computation affordable on a 1.07M-row training set, we drew a fixed 200,000-row sample and computed both Pearson and

Spearman correlation matrices on the 14 numeric features. The Pearson matrix captures linear correlation and is sensitive to extreme values, while Spearman uses rank correlation and is more robust to the heavy tails we already documented in Section 3.1. The first thing that stands out from both heatmaps is that the features fall into clearly visible blocks. Returns move together with their derivative versions, trade sizes move with their matching trade indicators, cancellation sizes move with their matching cancellation indicators, and the bid-side and ask-side variables each form their own correlated group. The 14 numeric features are therefore not 14 independent dimensions: there is real redundancy in the data, and a much smaller representation should be able to summarise normal behaviour without losing much information.

The differences between the Pearson and Spearman heatmaps are also informative. In several of the heavy-tailed columns, Spearman shows stronger correlations than Pearson, which means the variables move together in a consistent monotonic way but the relationship is not linear. This is consistent with what we saw in Section 3.1: a handful of extreme rows distort the Pearson correlation, while Spearman, which only uses ranks, sees through them. From a modelling point of view, this tells us that a model which only captures linear relationships between features (such as a Mahalanobis baseline) may underperform compared to a model that can learn non-linear structure (such as an autoencoder).

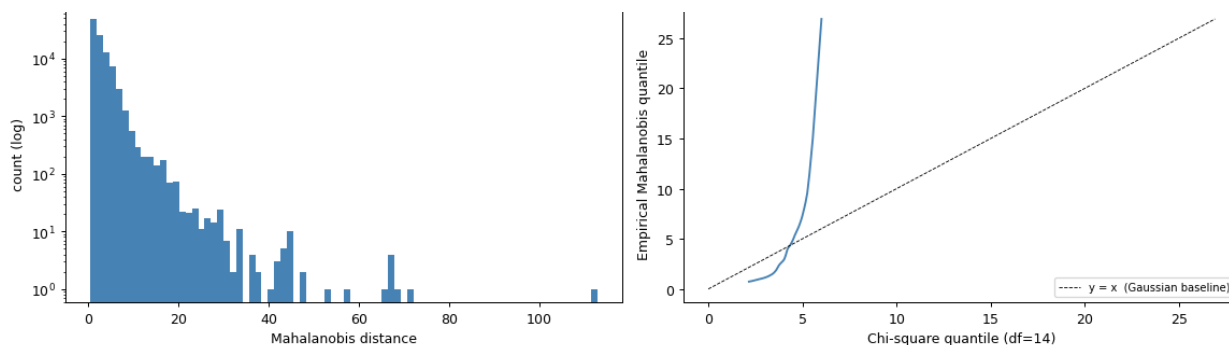


Figure 7: Mahalanobis Distance and QQ Plot

To make that comparison concrete, we computed Mahalanobis distance on a 100,000-row training sample. The Mahalanobis distance measures how far each row sits from the training mean after accounting for the covariance between features, so it is the natural multivariate generalisation of a z-score. If the normal data were close to a 14-dimensional Gaussian, the squared distances would follow a chi-square distribution with 14 degrees of freedom, and a QQ plot of the empirical distances against the chi-square reference would sit on the diagonal. Our QQ plot does not. At the median, the empirical Mahalanobis distance is 2.04, while a 14-dimensional Gaussian would predict around 3.65, so normal rows are actually more tightly packed around the mean than a Gaussian would expect. In the upper tail the picture flips. The empirical p99 is 12.16 against a Gaussian baseline of 5.40, and the gap keeps widening: at p99.9 the empirical distance reaches 26.91 against a Gaussian baseline of 6.01, and at p99.99

it reaches 47.96 against 6.53. The right tail of the joint distribution is much heavier than a multivariate Gaussian would allow.

The combined picture from this section is that the normal data has clear joint structure, but it is not joint-Gaussian. The block structure in the correlation matrices justifies a compressed representation of the feature space, which is what the autoencoder bottleneck in Section 5 ends up doing: the model can learn the bid/ask, return/derivative, and trade/indicator block relationships and use deviations from those relationships as an anomaly signal. The non-Gaussian tails are what tell us that Mahalanobis distance on its own is not a strong candidate model. A row with a large Mahalanobis distance is not necessarily anomalous, because normal rows themselves can travel much farther than a Gaussian would predict. We keep Mahalanobis in mind as a baseline reference, but the anomaly detectors built later in the report (Isolation Forest, autoencoder reconstruction, and the engineered z-score) are designed to handle the non-Gaussian structure that the QQ plot revealed.

3.5 Manipulation Pattern Fingerprints

Sections 3.1 to 3.4 told us what normal market behaviour looks like in this dataset. Section 3.5 asks the opposite question: if a manipulation pattern is present, what should it look like in our features? Rather than running more statistical tests, we approach this qualitatively, identifying which raw columns each pattern would reasonably affect and what the test data shows compared to train when we plot those columns. The three manipulation types we focus on are pump-and-dump, layering, and quote stuffing, which differ in market mechanics and therefore leave different fingerprints in the data. The table below acts as a roadmap for the rest of the section.

Pattern	Human description	Columns that react	Visual probe in notebook
Pump-and-dump	Coordinated price excursion followed by reversal	<i>ReturnBid/Ask1</i> , <i>DerivativeReturn*</i> , <i>Trade*Size</i>	Cumulative-return time series
Layering	Large displayed orders cancelled before they trade	<i>BidSize1</i> , <i>Cancelled*Size</i> , <i>Trade*Size</i>	(<i>TradeBidSize</i> , <i>CancelledBidSize</i>) scatter
Quote stuffing	Bursts of events with no trades	Inter-event Dt, <i>Trade*Indicator</i>	Train vs. test Dt overlay

We started with pump-and-dump (Figure 8). The pattern is well known in market microstructure: buying pressure pushes the price quickly up, then the position is unwound and the price falls back down. We do not have a raw price column in this dataset, but cumulative *ReturnBid1* over time acts as a shape proxy for price movement, so the suspicious signature is a sharp upward run followed by a quick reversal. The columns that should react are *ReturnBid1* and *ReturnAsk1* for

the directional movement, *DerivativeReturnBid1* and *DerivativeReturnAsk1* for the acceleration and reversal moments, and *TradeBidSize* and *TradeAskSize* for the elevated volume that usually accompanies these moves. We plotted cumulative *ReturnBid1* over the trading day for INTC, side by side for train and test. The most important thing to read from this plot is the shape of the cumulative curve over time, not any single return value. Suspicious areas would show sharp upward or downward excursions followed by a quick reversal. The implication for modelling is that a per-row anomaly score will not catch pump-and-dump on its own; the signal is sequential, so we will need rolling or windowed features.

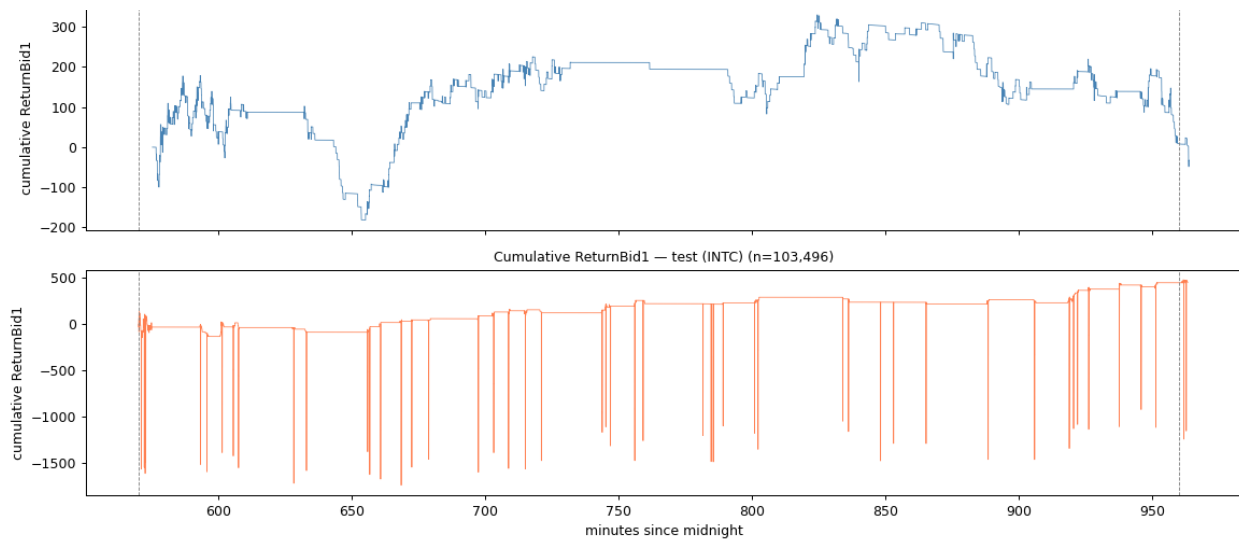


Figure 8: Pump and Dump (How Quick Changes in Bid Quantities Potentially Signal PaD)

Layering is structurally different. The strategy is to place large orders on one side of the book to create a false impression of demand or supply, then cancel those orders before they execute. The fingerprint in the data is a combination of two columns rather than an extreme single-column value: high cancellation activity (*CancelledBidSize*, *CancelledAskSize*) combined with low actual trading (*TradeBidSize*, *TradeAskSize*) on the same side of the book. We plotted *TradeBidSize* against *CancelledBidSize* on a 50,000-row sample from each split. In normal trading, these two quantities move together along a diagonal: large cancellations usually come with some trading activity. Layering would show up as points in the upper-left region of Figure 9, where cancellations are large but trading is small. The implication for modelling is that the right feature here is a per-row ratio (such as cancel-to-trade), which captures the joint imbalance without needing a time window.

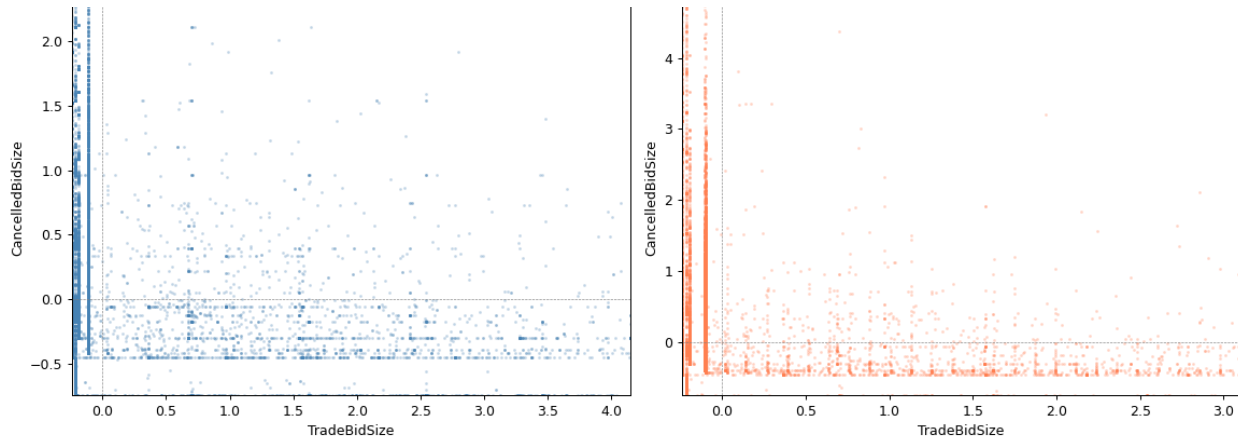


Figure 9: Layering

Quote stuffing is the third pattern, and it is the most timing-dependent of the three. The strategy is to flood the order book with many rapid order submissions and cancellations without any real intent to trade. The natural feature is the gap between consecutive events, computed per symbol after sorting by *TimeInMilliSecs*. We plotted the $\log_{10}$ inter-event delta-t distribution for train and test as an overlay (Figure 10), and pulled out percentile gaps to compare them numerically. At the lowest percentiles the gaps are identical ($p1 = 1$ ms in both splits), but the middle of the distribution does shift: test gaps are about half the size of train at $p25$ (1 ms versus 2 ms), three-quarters of train at $p50$ (15 ms versus 20 ms), and about two-thirds at $p75$ (160 ms versus 245 ms), before catching up again at $p99$ (3,873 ms versus 4,318 ms, a ratio of 0.90). So test events arrive faster on average, but the marginal distributions look broadly similar in shape. The implication for modelling is that quote stuffing is unlikely to be visible as a single-gap signal. The signal is more likely concentrated in short local bursts of compressed activity, which means we need rolling event-rate features or burst counts within a fixed window.

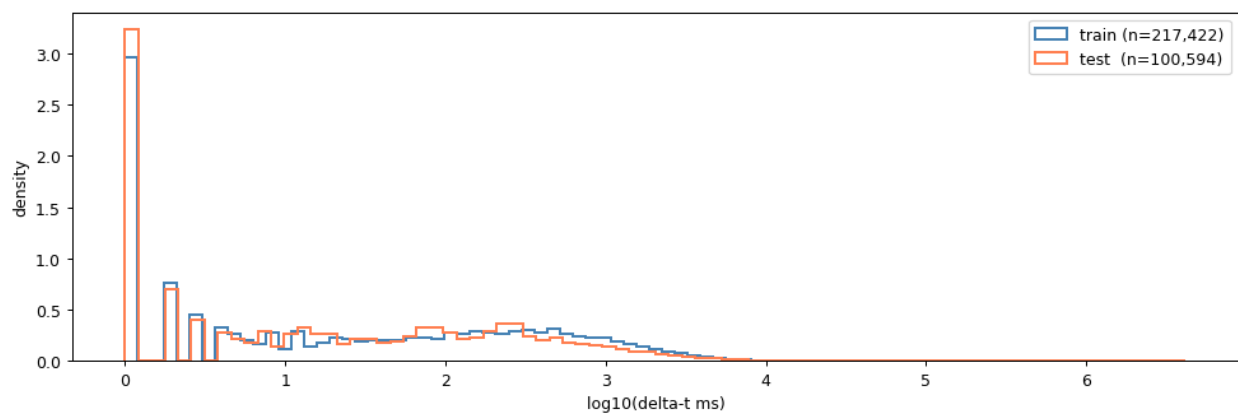


Figure 10: Quote Stuffing

Taken together, the three patterns push toward three different families of engineered features rather than a single uniform feature set. Pump-and-dump is a sequential pattern that needs rolling cumulative returns and reversal scores. Layering is a per-row joint pattern that needs simple ratios

between cancellation and trade size on the same side of the book. Quote stuffing is a burst pattern that needs windowed event-rate features instead of single inter-event gaps. This insight directly shapes the feature engineering in Section 4: the burst features in Section 4.1 target the quote-stuffing fingerprint through *OriginalSequenceNumber* grouping, the per-symbol z-scores in Section 4.2 give magnitude features the right per-stock baseline, and the magnitude and spread features in Section 4.3 capture the return-excursion and bid-ask divergence signals from pump-and-dump and layering.

3.6 EDA Synthesis

The EDA in Sections 3.1 to 3.5 consolidates into three high-level findings that drive every design decision in the rest of the report. Each finding below is paired with the evidence supporting it and the feature-engineering choice it forces in Section 4.

Finding 1: The data is pre-processed onto a near-centered, near-unit scale with heavy positive tails.

- Means cluster near zero and standard deviations near one, but extremes reach roughly plus or minus 25 to 50 (Section 3.1).
- The Mahalanobis QQ plot diverges sharply above the Gaussian reference at p99 and beyond (Section 3.4).
- Around twenty percent of rows in several trade and indicator columns sit on a single repeated "no event" value (Section 3.1).
- *Consequence for Section 4:* the scaling step uses RobustScaler (median and IQR) rather than StandardScaler, since the mean and standard deviation are pulled by the spikes and stretched by the tails (Section 4.5).

Finding 2: The test set differs from train along multiple confounded axes (date, symbol mix, intraday shape) simultaneously.

- TradeBidSize reaches KS D = 0.65 against test, with five other features above 0.40 (Section 3.2).
- The test set is dominated by INTC and MSFT at close to 70 percent combined, while train and validation hold each symbol at roughly 17 to 23 percent (Section 3.3).
- Test sits on a single trading day while train and validation span multiple dates (Section 3.1).
- *Consequences for Section 4:* z-scores are computed per symbol so that each row's anomaly score uses the right baseline (Section 4.2), especially for *BidSize1* with its 3.15 times per-symbol std ratio. The unresolved confounds also push the project toward an unsupervised modelling approach in Section 5, since direct train-vs-test feature comparisons would be contaminated.

Finding 3: The three manipulation types live in three different feature families: per-row, joint, and sequential.

- Pump-and-dump is sequential, visible in the shape of cumulative returns over time (Section 3.5).
- Layering is joint, visible in the relationship between cancellation size and trade size on the same side of the book (Section 3.5).
- Quote stuffing is bursty, visible in the rate of events within a short window rather than in a single inter-event gap (Section 3.5).
- *Consequence for Section 4:* the engineered feature set spans all three families. Burst features in Section 4.1 capture quote stuffing through *OriginalSequenceNumber* grouping, magnitude and spread features in Section 4.3 capture pump-and-dump amplitude and bid-ask divergence under layering, and per-symbol z-scores in Section 4.2 keep every signal on a fair per-stock baseline. The full 37-feature matrix is described in Section 4.4.

4. Feature Engineering

After preparing the data, we expanded the original 14 limit order book features into a 37-feature matrix. The goal was to transform the raw order-book columns into features that better captured the manipulation signals identified during the EDA, mainly burst activity from **Section 3.5**, per-symbol differences from **Section 3.3**, and joint bid-ask behavior from **Section 3.4**.

In this section, we created five groups of features: the original raw LOB features, burst-order features, per-symbol z-scores, magnitude and spread features, and scaled features used for model training. The purpose and construction of each feature group are explained in the following subsections.

4.1 Burst Features

The first group of engineered features was based on *OriginalSequenceNumber*. This column was important because it helped capture burst-like order activity, which is closely related to quote stuffing. In quote stuffing, suspicious behavior does not necessarily come from one isolated order. Instead, the signal comes from many rapid order events linked to the same underlying sequence.

To extract this structure, we split *OriginalSequenceNumber* on the hyphen and kept the first token as the base sequence ID. For example, a value such as "16120456-000001-026" was treated as belonging to the base sequence 16120456. We then counted how many rows shared the same base sequence ID within the same split. This count became the *burst_size*. A row with a burst size of 1 represents a standalone event. A row with a larger burst size belongs to a repeated sequence of events generated by the same base order. This matters because abnormal bursts can indicate unusually intense activity in the order book, which is one possible footprint of quote stuffing.

The EDA showed that burst size was one of the strongest signals in the entire project. In particular, rows with burst sizes between 10 and 49 had much larger absolute return movements than single-order rows:

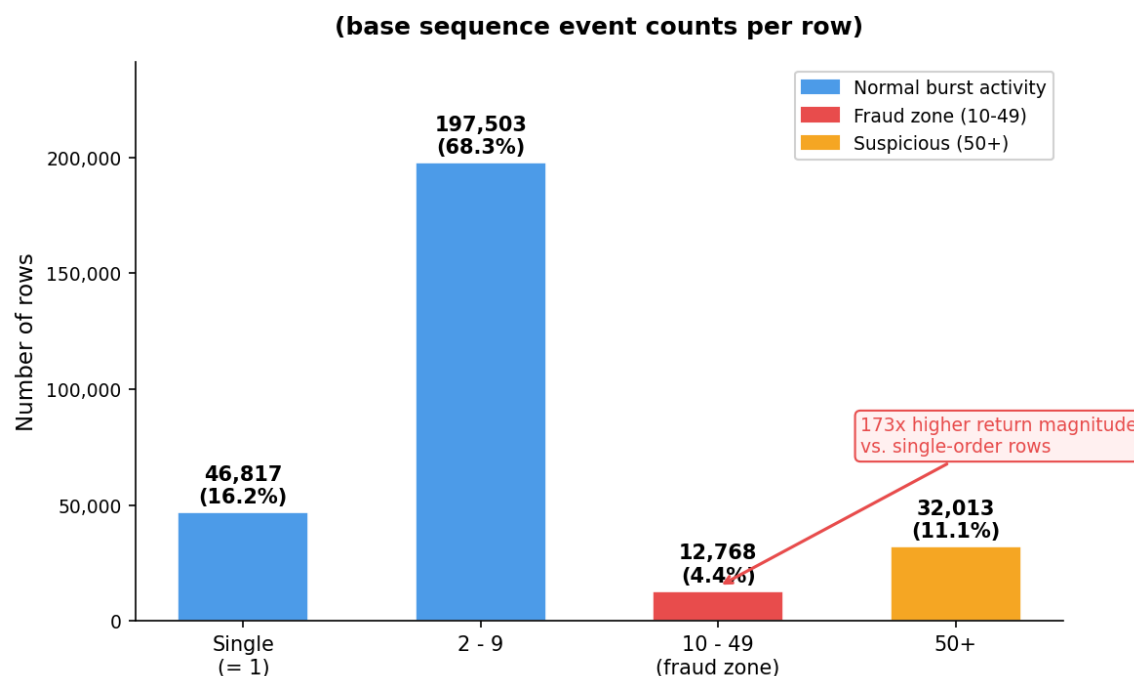


Figure 11: Burst Size Distribution

From these results, the main takeaway was that burst activity carried a strong anomaly signal, but not in a perfectly linear way. The 10–49 range stood out the most, with a mean absolute ReturnBid1 that was 173× higher than the single-order baseline (Figure 11). Because of that, we did not want to represent burst size only as one continuous variable.

To capture both the general burst effect and the specific high-risk ranges, we created three burst features:

- *burst_log* = $\log(1 + \text{burst_size})$, which captures burst size as a continuous feature while reducing the effect of very large values.
- *burst_10_49*, a binary feature equal to 1 when the burst size falls in the 10–49 range.
- *burst_50plus*, a binary feature equal to 1 when the burst size is 50 or higher.

These features were especially important in the final z-score detector. The *burst_10_49* feature received the highest weight in the anomaly formula because it was directly supported by the 173× return-magnitude ratio observed in the EDA.

4.2 Per-Symbol Z-Scores

The second group of engineered features was based on per-symbol z scores. These features measure how far each observation is from the normal behavior of the same stock, instead of comparing all rows to one general market-wide average;

We chose this approach based on the EDA. Some variables had very different ranges depending on the symbol, meaning that the same value could look normal for one stock and unusual for another. For example, a *BidSize1* value of 2.0 was much more extreme for AAPL than for GOOG. A pooled baseline would miss this difference and could make the anomaly scores less reliable.

To avoid this, we computed the mean and standard deviation separately for each symbol using only the normal pool. For each row, feature, and symbol, we calculated the absolute z-score:

$$|z_{i,f,s}| = \frac{|x_{i,f} - \mu_{f,s}|}{\sigma_{f,s}}$$

where $x_{i,f}$ is the value of feature f for row i , $\mu_{f,s}$ is the normal-pool mean of feature f for symbol s , and $\sigma_{f,s}$ is the corresponding standard deviation.

We computed one z-score for each of the 14 raw numeric features. We also created two aggregate z-score features:

- *max_zscore*, which captures the largest individual deviation across all raw features for a row.
- *mean_zscore*, which captures the average level of abnormality across the row.

With these 16 z-score features, each row was evaluated relative to the normal behavior of its own stock. This made the engineered feature set more sensitive to truly unusual observations, while reducing the risk of confusing normal symbol-level differences with actual anomalies.

4.3 Magnitude and Spread Features

The third group of engineered features captured raw magnitude and bid-ask imbalance. These features were motivated mainly by **Section 3.4**, where the joint feature analysis showed that bid-side and ask-side variables normally move in a structured way rather than independently. Because of this, a strong divergence between the two sides of the book can be treated as a useful stress signal.

We created four magnitude and spread features:

- *max_abs_feat*, which captures the maximum absolute value across the 14 raw LOB features for a row. This gives a simple measure of the strongest raw deviation in the row, regardless of which feature caused it.
- *abs_RetBid1*, which captures the absolute return movement on the bid side.
- *abs_RetAsk1*, which captures the absolute return movement on the ask side.
- *return_spread* = $|ReturnBid1 - ReturnAsk1|$, which captures the difference between bid-side and ask-side returns.

The return magnitude features are useful because some manipulation patterns, especially pump-and-dump behavior discussed in **Section 3.5**, can appear through unusually large return movements. The `return_spread` feature is useful for a different reason. As shown in **Section 3.4**, the two sides of the book usually co-move under normal conditions. When *ReturnBid1* and *ReturnAsk1* move far apart, this can indicate pressure or imbalance in the order book. In the context of manipulation, this type of divergence may be linked to spoofing or layering, where one side of the book is artificially affected by abnormal order placement or cancellation behavior.

These features helped preserve simple but important signals that would be weakened if we relied only on normalized z-scores. The z-score features measure how unusual a value is relative to the stock's normal distribution, while the magnitude and spread features keep direct information about large raw movements and bid-ask separation.

4.4 The 37-Feature Set

After feature engineering, each row was represented using 37 features. The final feature set combined raw market information with engineered signals designed to capture the main manipulation patterns.

<i>Feature group</i>	<i>Count</i>	<i>Features</i>
<i>Raw LOB features</i>	<i>14</i>	<i>ReturnBid1, ReturnAsk1, derivative returns, bid/ask sizes, trade sizes, cancellation sizes, and activity indicators</i>
<i>Per-symbol z-scores</i>	<i>14</i>	<i>One z-score for each raw numeric feature</i>
<i>Z-score aggregates</i>	<i>2</i>	<i>max_zscore, mean_zscore</i>
<i>Magnitude and spread features</i>	<i>4</i>	<i>max_abs_feat, abs_RetBid1, abs_RetAsk1, return_spread</i>
<i>Burst features</i>	<i>3</i>	<i>burst_log, burst_10_49, burst_50plus</i>
<i>Total</i>	<i>37</i>	<i>Full engineered feature matrix</i>

This feature set was designed to connect directly to the three main manipulation patterns in the project. Pump-and-dump behavior is mainly captured through return magnitude and z-score extremity. Spoofing and layering are captured through cancellation-related features and bid-ask imbalance. Quote stuffing is captured through the burst-order features derived from `OriginalSequenceNumber`.

4.5 Scaling

Before using the 37-feature matrix in the modelling stage, we applied `RobustScaler`. The scaler was fitted on the normal pool only and then applied to both the normal-pool feature matrix and the

test feature matrix. This kept the transformation consistent while ensuring that no information from the test set influenced the fitted scaling parameters.

RobustScaler centers each feature using the median and scales it using the interquartile range:

$$x_{scaled} = \frac{x - median(X_{normal})}{IQR(X_{normal})}$$

Where:

$$IQR = Q_{75} - Q_{25}$$

We chose this scaling method over standard scaling because of the distributional patterns identified in **Section 3.1**. StandardScaler uses the mean and standard deviation, but both are sensitive to extreme values. In this dataset, several variables had heavy positive tails and mass spikes around repeated “no event” values. As a result, the mean and standard deviation would not represent the typical spread of the data very well. RobustScaler was more appropriate because the median and IQR are less affected by extreme observations. The median captures the center of the distribution without being pulled strongly by large values, while the IQR measures the spread of the middle 50% of observations. This makes the scaling parameters more representative of typical normal trading behavior, rather than being distorted by rare large movements or repeated mass-spike values.

This scaling step was especially important as a preparation step for the modelling done in the upcoming sections. It ensured that the engineered features were placed on a comparable scale, while still preserving the no-leakage structure of the pipeline by fitting the scaler only on the normal pool.

5. Simple Supervised and Unsupervised Models

5.1 The Semi-Supervised Challenge

After the feature engineering step in Section 4, we had a 37-column matrix that summarises every row of the dataset in terms suitable for modelling. The natural next step is to train a standard supervised classifier on this matrix and have it predict whether each test row is normal or anomalous. The problem is that this is not actually possible with the data as given. Both the training set and the validation set are 100% normal: every row in train (1,070,022 rows) and every row in val (467,939 rows) carries *TARGET* = 0, as we confirmed in the data cleaning step of Section 2. The test set contains the hidden mix of normal and anomalous rows, but its labels are not available to us during model fitting. We therefore have no positive class to train any supervised model against.

This matters because a supervised classifier learns by contrasting examples of different classes. Logistic regression fits a sigmoid that separates positive from negative rows. Random Forest splits the feature space based on the labelled outcome at each node. A linear SVM finds the hyperplane

that maximises the margin between the two classes. All three of these methods assume that the training data contains both positive and negative examples. If we hand them a dataset where every row is labelled 0, the optimal model is the trivial one that always predicts 0, which would score zero on the leaderboard because it cannot distinguish between rows at all. An unsupervised path fits this setup more naturally, since methods like the Dense Autoencoder in Section 5.6 can learn directly from normal data and score test rows by how far they deviate from it. But we also wanted to test the standard supervised models we covered in class, both to compare their performance against the unsupervised approach and to understand how much of the anomaly signal is reachable using familiar methods.

To make supervised training possible, we used synthetic anomaly injection. The idea is to take a small subset of normal training rows, deliberately perturb them in a way that mimics one of the manipulation patterns from Section 3.5, label those perturbed rows as $TARGET = 1$, and combine them with unperturbed normal rows to build a labelled dataset that the supervised models can actually fit. This is a workaround rather than a clean solution: the synthetic anomalies only cover the magnitude-amplification fingerprint of pump-and-dump, which means the resulting supervised models will be biased toward catching that one pattern and may miss layering and quote stuffing. This trade-off is what eventually pushes us back to unsupervised methods later in Section 5. Section 5.2 explains exactly how we generated these synthetic labels.

5.2 *Synthetic anomaly injection*

Since the data does not give us any labelled fraud examples, we built our own. The high-level recipe is straightforward: take a fixed sample of normal rows from the training pool, perturb a subset of them in a way that mimics a real manipulation pattern, treat those perturbed rows as the positive class, and combine them with unperturbed normal rows to form a labelled dataset. This lets us train the supervised models in Sections 5.3 through 5.5 even though the underlying training data contains no positive class. The whole process is wrapped in a single function in the notebook, `inject_synthetic_anomalies`, and uses fixed random seeds so the labelled dataset is reproducible.

The mechanic targets the pump-and-dump fingerprint identified in Section 3.5. We sampled 20,000 rows uniformly at random from the normal pool and amplified their return-related columns. For each sampled row, we drew a scaling factor from $\text{Uniform}[10, 50]$ and multiplied *ReturnBid1* and *DerivativeReturnBid1* by that factor, then multiplied *ReturnAsk1* and *DerivativeReturnAsk1* by the same factor times a separate uniform draw in $[0.8, 1.2]$. The asymmetric ask-side scaling preserves the structural fact that the two sides of the book usually move together but not perfectly in lockstep, which we noted in Section 3.4. The range $[10, 50]$ was chosen to produce synthetic returns large enough to register as clearly anomalous at the z-score level: Section 3.1 showed that normal return columns reach roughly plus or minus 25 to 50 in their most extreme rows, so a 10-fold to 50-fold scaling of a typical return is enough to push the synthetic rows into the same magnitude band as the rare normal extremes, while the matching z-score columns mark them as far from the per-symbol baseline. We then labelled those 20,000 perturbed rows as $TARGET = 1$

and combined them with 100,000 unmodified normal rows labelled *TARGET* = 0, producing a 120,000-row dataset with a fraud rate of one in six. This ratio gives the supervised models enough positive examples to learn a decision boundary without making the positive class dominant.

A small but important detail is that we did not stop after perturbing the raw columns. The perturbed rows were passed back through the full feature-engineering pipeline from Section 4, so all 37 features were recomputed from the new values. This matters because most of our features are derived: the per-symbol z-score *z_ReturnBid1* depends on the raw *ReturnBid1*, *abs_RetBid1* is the absolute value of the perturbed return, *max_abs_feat* is the maximum absolute feature across the row, *return_spread* is the difference between bid-side and ask-side returns, and so on. If we had perturbed only the raw return columns and kept the original z-scores and aggregate features, the labelled dataset would carry an internal inconsistency that the models could potentially exploit in unrealistic ways. By re-running the engineering step, we ensure that a 30-times perturbation in *ReturnBid1* propagates correctly to its z-score, its absolute value, and the return-spread between bid and ask sides.

We also fit a separate *RobustScaler* on this labelled mix rather than reusing the scaler from Section 4.5, since the labelled dataset's distribution is different from the normal-only pool used elsewhere in the project. The new scaler is fit on the 120,000-row labelled matrix and then applied to the test matrix when we score it with each supervised model. This keeps the scaling consistent between training and inference for the supervised pipeline while preserving the no-leakage rule, since no test data participates in fitting the scaler.

The trade-off is worth stating clearly because it shapes everything that follows. This injection only simulates the magnitude-amplification fingerprint of pump-and-dump. Layering, which lives in the joint relationship between cancellation size and trade size, is not represented in the synthetic anomalies. Quote stuffing, which lives in the burst structure of event sequences, is also absent. Any supervised model fit on this labelled dataset will therefore be biased toward catching return-magnitude spikes and may miss the other two manipulation types entirely. This is the structural ceiling of the supervised approach and explains why even our best supervised model in Sections 5.3 through 5.5 underperforms the unsupervised Dense Autoencoder in Section 5.6, despite the autoencoder using the same 37-feature matrix.

5.3 Model 1: Logistic Regression

Logistic regression is the natural starting point for the supervised path. It is the simplest linear classifier, it converges quickly on a dataset of this size, and the fitted coefficients can be read directly to understand what the model has learned. We fit *sklearn*'s *LogisticRegression* on the 120,000-row labelled matrix from Section 5.2, with *max_iter* = 500 and the default L2 penalty (*C* = 1.0). The inputs were the 37 engineered features scaled by the supervised *RobustScaler* defined in Section 5.2, and the target was the binary fraud label (0 = normal, 1 = synthetic). After fitting,

we used `predict_proba` on the scaled test matrix to assign each test row a fraud probability in `[0, 1]`, which became our submission.

The top five features by absolute coefficient size tell us what the model leaned on:

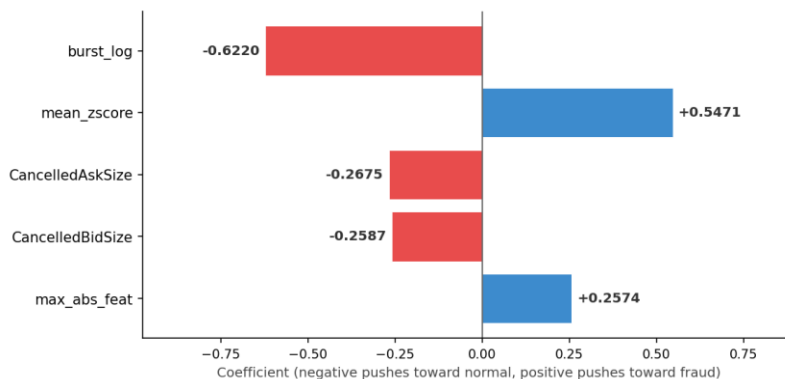


Figure 12: Most Important Features LR

Two of these coefficients have the sign we would expect. *mean_zscore* carries a positive coefficient, which means rows that deviate more on average across all per-symbol z-scores are pushed toward the fraud class. *max_abs_feat*, the largest raw absolute deviation across the row, is also positive. Both align with the synthetic perturbation: when we multiply *ReturnBid1* by a factor of 30, the row's *mean_zscore* and *max_abs_feat* both shoot up, and the model correctly learns that those columns are fraud signals.

The other three coefficients are more surprising. *burst_log*, *CancelledAskSize*, and *CancelledBidSize* all carry strongly negative coefficients, which means the model has learned that high burst activity and large cancellation sizes are signs that a row is NOT fraud. This is the opposite of what the EDA in Section 3.5 told us: bursts are the natural footprint of quote stuffing, and large cancellations are the natural footprint of layering. The reason follows directly from how the synthetic labels were built in Section 5.2. Our injection only perturbed the four return columns; we did not touch *burst_log*, *CancelledAskSize*, or *CancelledBidSize*. So inside the labelled dataset, the fraud class is a sample of normal rows with amplified returns but otherwise typical burst and cancellation values, while the normal class includes some rows with naturally high burst or cancellation activity. The LR fits to that contrast and concludes that high burst means non-fraud, which is exactly backwards relative to real layering and quote stuffing.

On the Kaggle public leaderboard, logistic regression scored an AUROC of **0.38342**. This is well below the 0.5 random baseline, which means the model's fraud scores are anti-correlated with the actual test labels rather than uncorrelated. The implication is that the real test anomalies are not predominantly pump-and-dump cases. If they were, the positive coefficients on *mean_zscore* and *max_abs_feat* would carry the day and the AUROC would land above 0.5. Instead, a large share of the test anomalies must be layering and quote-stuffing rows, which carry exactly the kind of high burst and high cancellation signals that our model has learned to mark as normal.

Mechanically, inverting the model's scores would give an AUROC of 0.6158, which is more than the random baseline but still well below what we eventually achieve with the unsupervised detectors in Section 5.6 and beyond.

This result confirms the structural ceiling we flagged in Section 5.2. Logistic regression did not just fail to detect the manipulation types that were absent from the synthetic injection; it actively learned to mark them as normal. The model is not under-fitting or over-fitting in the usual sense, it is fitting a different problem than the one Kaggle is grading us on. The Random Forest in the next subsection runs into the same structural issue, but its non-linear inductive bias gives it more room to compensate.

5.4 Model 2: Random Forest

Random forest is the natural next step after logistic regression because it relaxes the linear assumption. Instead of learning a single global decision boundary, the model averages over many shallow decision trees and can capture interactions between features that a linear model cannot. We fit sklearn's `RandomForestClassifier` on the labelled dataset from Section 5.2 with `n_estimators = 100`, `max_depth = 15`, and `min_samples_leaf = 20`. The depth limit and the minimum-leaf-size constraint together keep individual trees from memorising the training set, which matters here because the synthetic positive class is small enough (20,000 rows) to be over-fittable. Unlike logistic regression, random forest does not benefit from feature scaling, so we fit it directly on the unscaled 37-feature matrix.

The top five features by importance tell a tighter story than the LR coefficients:

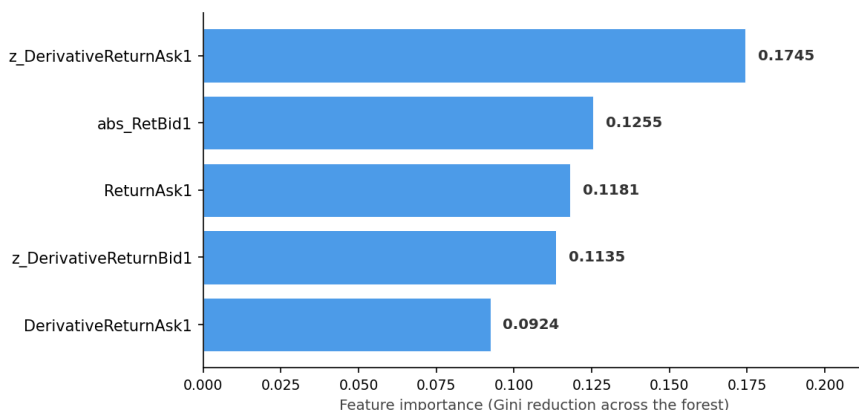


Figure 13: Most Important Features RF

All five are return-related, and together they account for about 62% of the total feature importance. The model has effectively built its decision boundary out of the four return columns and their per-symbol z-scores, with one extra magnitude feature (*abs_RetBid1*) included. None of the burst, cancellation, or trade-side features make the top five, which is a sharp contrast with logistic regression, where *burst_log* carried the largest absolute coefficient.

This concentration is the direct mechanical consequence of the synthetic injection. The fraud rows in the labelled dataset differ from the normal rows in exactly four raw columns (*ReturnBid1*, *ReturnAsk1*, *DerivativeReturnBid1*, *DerivativeReturnAsk1*) and the per-symbol z-scores that depend on them. A tree-based model trying to separate the two classes will naturally pick the splits on those columns, since splitting on them produces clean partitions of the labelled data while splitting on *burst_log* or *CancelledBidSize* does not. The non-return features do appear inside the forest in deeper splits, but their overall importance is low because the model rarely needs them once the return-based splits have done the work. This is a passive ignoring rather than the active anti-learning we saw in logistic regression. Random forest's feature importances are non-negative by construction, so columns that fail to separate the classes simply contribute little; they do not actively push true positives toward the normal class the way negative LR coefficients did.

On the Kaggle public leaderboard, random forest scored an AUROC of **0.85103**. This is a substantial improvement over logistic regression's 0.38342 and well above the random baseline of 0.5, but it still falls just short of the official competition baseline of 0.88. The interpretation is consistent with the structural ceiling discussion from Section 5.2. By relying entirely on return-magnitude features, the forest scores the pump-and-dump component of the test set correctly while assigning roughly neutral scores to the layering and quote-stuffing rows that dominated logistic regression's failure mode. It does not detect those other patterns, but it does not actively mis-rank them either. The model behaves like a strong pump-and-dump detector and a mediocre catch-all anomaly score: well above the random line, well below what a model that could see the burst and cancellation patterns would achieve.

The gap between the two supervised models is informative in its own right. Logistic regression and random forest were trained on the same 120,000-row labelled dataset and the same 37-feature matrix. The only real difference between them is the inductive bias: LR forces a single global linear boundary, while RF allows many local non-linear splits. That bias difference alone is enough to turn a sub-random model into one that lands close to the leaderboard baseline. The remaining gap to truly competitive scores comes from the synthetic injection itself, which the linear SVM in the next subsection inherits in exactly the same way.

5.5 Model 3: Linear SVM

A linear SVM gives us a third inductive bias on the same supervised setup. Where logistic regression fits a sigmoid by maximising the log-likelihood and random forest builds a decision boundary out of many local splits, an SVM finds the single linear hyperplane that maximises the margin between the two classes. We use the LinearSVC implementation rather than SVC with an RBF kernel because the kernelised version does not scale comfortably to a 120,000-row training matrix in the standard sklearn implementation, while LinearSVC fits in a few seconds. The model is configured with $C = 1.0$, `max_iter = 3000`, and `dual = 'auto'` so sklearn picks the right primal/dual formulation for our matrix shape, with `random_state = 42` for reproducibility. We fit it on the same scaled 120,000-row labelled matrix that logistic regression used.

LinearSVM does not return probabilities; it returns a signed margin via `decision_function`, which is the distance from the hyperplane in feature space. Positive values indicate the model classifies the row as fraud and negative values indicate normal, with magnitude measuring how far the row sits from the decision boundary. These margins are unbounded and arbitrary in scale, so they cannot be submitted directly. We apply min-max normalisation to compress the test margins into the $[0, 1]$ range that Kaggle expects, mapping the smallest margin to 0 and the largest to 1 and scaling everything else proportionally in between. This preserves the ranking of test rows by margin while making the output a valid submission. Because Kaggle scores AUROC, which depends only on ranking, the min-max step does not change the model's evaluated performance.

The top five features by absolute coefficient size are:

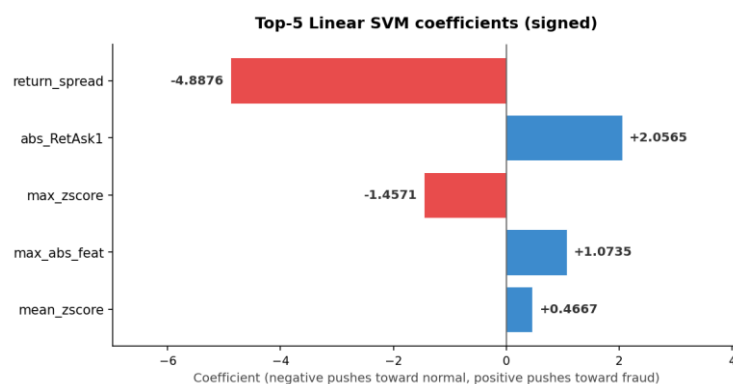


Figure 14: Most Important Features SVM

The signs are a mix of expected and unexpected. *abs_RetAsk1*, *max_abs_feat*, and *mean_zscore* all carry positive coefficients, which matches the EDA-driven intuition that rows with extreme returns or large overall deviation should be classed as fraud. *return_spread* and *max_zscore*, on the other hand, carry strongly negative coefficients, which is the opposite of what we would expect: synthetic anomalies should both widen the bid-ask return spread (because the asymmetric scaling factor introduces extra noise between the two sides) and inflate the max per-symbol z-score (since the perturbation directly amplifies the return columns whose z-scores enter the max). The most likely explanation is that the linear-SVM hinge loss places different weight on different parts of the training distribution than the LR log-loss did. The resulting hyperplane appears to use *return_spread* and *max_zscore* as correction terms that reduce the model's score for rows that score high on the other magnitude features at the same time, rather than as direct fraud indicators. With the correlations between magnitude features baked into the labelled dataset, this kind of coefficient redistribution is what produces large coefficients with non-intuitive signs.

On the Kaggle public leaderboard, the linear SVM scored an AUROC of **0.64425**. This sits between logistic regression at 0.38342 and random forest at 0.85103, which lines up with where the model fits in the spectrum of supervised options. The SVM shares the linear assumption that broke logistic regression on this dataset, so it is vulnerable to the same kind of sign-flip behaviour on features the synthetic injection did not perturb. But its hinge-loss objective gives a different

coefficient assignment than LR's log-likelihood objective, and that difference is enough to flip just one or two of the most damaging negative coefficients from LR back toward the right side of the decision boundary. The net result is a model that is no longer below random, but is also nowhere near competitive with the leaderboard baseline.

Across the three supervised models, the same pattern keeps appearing. The synthetic injection in Section 5.2 makes the labelled training distribution different from the test distribution in a way that any supervised classifier inherits. Logistic regression amplifies the mismatch and lands below random. Linear SVM partially compensates and lands above random but below baseline. Random forest's non-linear inductive bias avoids the worst of the mismatch and lands near baseline, but still cannot beat it. None of the three reach a Kaggle score that would be worth submitting as a final answer. This is the empirical confirmation of the structural ceiling we identified in Section 5.2, and it is exactly the reason the project pivots to unsupervised detection in Section 5.6, which uses the same 37-feature matrix but learns directly from the normal pool without any synthetic labels at all.

5.6 Model 4: Dense Autoencoder

Next, to move beyond the limitations of the supervised models, we switched to unsupervised anomaly detection. This was necessary because the supervised models depended on synthetic fraud labels, which only captured one type of abnormal behavior: return-magnitude spikes. Since the real test anomalies could also include spoofing, layering, and quote-stuffing patterns, we needed models that could learn from normal data directly instead of relying on artificial fraud examples. The first unsupervised model we tested was a Dense Autoencoder. This model is an extension of the simple autoencoder studied in class, where the network compresses the input into a smaller hidden representation and then reconstructs it back to the original feature space. In our case, the same idea was applied to the full 37-feature matrix: the model learns how normal order-book rows should look, then assigns higher anomaly scores to rows that it cannot reconstruct well. This makes the autoencoder useful for our problem because it does not need labelled fraud examples. It only needs normal data, which is exactly what we have in the train and validation sets. It can also capture non-linear relationships between features, meaning it may detect unusual combinations of returns, trade sizes, cancellations, and burst patterns even when no single feature is extreme on its own.

The following section explains the model mathematically as an extension of the autoencoder architecture covered in class, and then shows how we used reconstruction error to score anomalies.

Architecture:

In the example we saw in class, the model takes two input features, compresses them into one hidden representation and then reconstructs the original two features. The main idea is the same here, but our model uses the full 37-feature matrix created during feature engineering.

Our autoencoder compresses the 37-feature input into an 8-dimensional bottleneck representation, then reconstructs it back to 37 features: $\text{Input (37)} \rightarrow \text{Dense(20, ReLU)} \rightarrow \text{Dense(8, ReLU)} \rightarrow \text{Dense(20, ReLU)} \rightarrow \text{Output (37, linear)}$

Before settling on this architecture, we considered whether a simpler vanilla autoencoder with a single hidden layer (37 to 8 to 37) would have been enough. We chose the multi-layer dense version for three reasons. First, a single-hidden-layer autoencoder with linear activations is mathematically equivalent to PCA, so it can only capture linear feature relationships. Section 3.4 showed that the data has clear non-linear structure (visible in the differences between Pearson and Spearman correlations in the heavy-tailed columns), so we wanted a model that could learn relationships beyond what PCA captures. Second, the EDA also showed a clear block structure across the bid-side, ask-side, return, and trade groups. Compressing 37 features straight down to 8 in one step would force the model to find all those latent factors at once, while the intermediate 20-unit layer lets the network strip the obvious linear redundancies first and use the 8-unit bottleneck for the harder latent structure. Third, because the anomaly score is the reconstruction error, the wider the gap between how well the model reconstructs normal rows and anomalous rows, the better the AUROC. A multi-layer dense network reconstructs the normal manifold more accurately than a single-bottleneck alternative, which sharpens that gap.

Returning to the mathematical motivation, the first part of the network is the encoder:

$$37 \rightarrow 20 \rightarrow 8$$

The encoder maps the original feature vector into a compressed representation:

$$\text{enc}: \mathbb{R}^{37} \rightarrow \mathbb{R}^8$$

The second part is the decoder:

$$8 \rightarrow 20 \rightarrow 37$$

The decoder maps the compressed representation back into the original feature space:

$$\text{dec}: \mathbb{R}^8 \rightarrow \mathbb{R}^{37}$$

So the full autoencoder can be written as:

$$\hat{\mathbf{x}} = \text{dec}(\text{enc}(\mathbf{x}))$$

The key part of the architecture is the bottleneck layer with only 8 units. This bottleneck prevents the model from simply copying the input directly and forces it to learn a compressed representation of normal rows. Since the model is trained only on the normal pool, the idea is that normal observations should pass through this bottleneck and reconstruct well, while unusual observations should reconstruct less accurately.

For a general layer l , the model first applies a linear transformation:

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

Then it applies the ReLU activation function:

$$\mathbf{a}^{(l)} = \text{ReLU}(\mathbf{z}^{(l)}) = \max(0, \mathbf{z}^{(l)})$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ is the original input row. The final output layer uses a linear activation, which means it directly reconstructs the original feature values:

$$\hat{\mathbf{x}} = W^{(4)} \mathbf{a}^{(3)} + \mathbf{b}^{(4)}$$

Training

During training, the autoencoder uses the normal feature matrix as both the input and the target. In code, this is written as: `model.fit(X_normal, X_normal)`

This means the model is not trying to predict a label. It is trying to reproduce each normal row as accurately as possible. The training loss is the mean squared reconstruction error:

$$L = \frac{1}{N} \sum_{i=1}^N \frac{1}{F} \sum_{f=1}^F (\hat{x}_{i,f} - x_{i,f})^2$$

where N is the number of normal training samples, $F = 37$ is the number of features, $x_{i,f}$ is the original value of feature f for row i , and $\hat{x}_{i,f}$ is the reconstructed value.

This is the same MSE idea from Chapter 11, but generalized from the small $2 \rightarrow 1 \rightarrow 2$ example to a 37-feature architecture. The model learns by making the reconstructed values as close as possible to the original values for all normal training rows.

We trained the model on the normal pool using the Adam optimizer, a learning rate of 0.001, a batch size of 1024, and 50 epochs. Since the model only sees normal rows during training, it should become good at reconstructing normal behavior. However, rows with patterns that were not present in the normal pool should be harder to reconstruct.

Backpropagation

Training is done through backpropagation, following the neural network logic covered in class. The model first computes the reconstruction error, then calculates how much each weight and bias contributed to that error. It then updates the parameters in the direction that reduces the loss.

For the output layer, the gradients can be written as:

$$\frac{\partial L}{\partial W^{(4)}} = \frac{1}{N} (\hat{X} - X)^T \mathbf{a}^{(3)}$$

$$\frac{\partial L}{\partial \mathbf{b}^{(4)}} = \frac{1}{N} \sum_i (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

For each hidden layer l , the error is passed backward using the chain rule:

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot \text{ReLU}'(\mathbf{z}^{(l)})$$

where:

$$\text{ReLU}'(z) = \mathbf{1}[z > 0]$$

and $\odot$ means element-wise multiplication.

The gradients for the weights and biases in layer l are:

$$\frac{\partial L}{\partial W^{(l)}} = \frac{1}{N} (\delta^{(l)})^T \mathbf{a}^{(l-1)}$$

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \frac{1}{N} \sum_i \delta_i^{(l)}$$

Then the weights are updated using gradient descent with learning rate ϵ :

$$W^{(l)} \leftarrow W^{(l)} - \epsilon \cdot \frac{\partial L}{\partial W^{(l)}}$$

In simple terms, the autoencoder repeatedly adjusts its weights so that normal rows can pass through the bottleneck and be reconstructed with low error.

Anomaly Scoring

After the autoencoder is trained, we use reconstruction error as the anomaly score. For each test row i , the reconstruction error is:

$$e_i = \frac{1}{F} \sum_{f=1}^F (\hat{x}_{i,f} - x_{i,f})^2$$

This is the same MSE formula used during training, but now it is evaluated on test rows. If a row is normal, the model should reconstruct it well, so e_i should be small. If a row is anomalous or fraudulent, the model may not reconstruct it well because that type of feature pattern was not learned during training. In that case, e_i should be larger.

Finally, the reconstruction errors are min-max normalized to the range $[0, 1]$:

$$\text{score}_{AE}(i) = \frac{e_i - \min_j e_j}{\max_j e_j - \min_j e_j + \varepsilon}$$

The small ε is added to avoid division by zero. After normalization, a higher $\text{score}_{AE}(i)$ means the row has a larger reconstruction error and is therefore more likely to be anomalous.

Result and Interpretation

The Dense Autoencoder achieved a public AUROC of **0.95785** on Kaggle, making it the strongest model among the first four models tested. It outperformed the supervised models because it was not limited by the synthetic anomaly injection process. The supervised models mainly learned the return-magnitude pattern created during injection, while the autoencoder learned directly from the normal pool.

This made the autoencoder more flexible than the supervised baselines. It could assign high scores to rows with unusual feature combinations, not only rows with large return spikes. However, it was still not the strongest individual detector overall, which is why we later compared it with Isolation Forest and the z-score detector.

5.7 Initial Kaggle results

To recapitulate the results we obtain, we simply are providing a table and plot to visualize the slow improvement in our model building process.

Model	Type	Score on Kaggle
Logistic Regression	Supervised	0.38342
Random Forest	Supervised	0.85103
SVM (Linear)	Supervised	0.64425
Dense Autoencoder	Unsupervised	0.95785

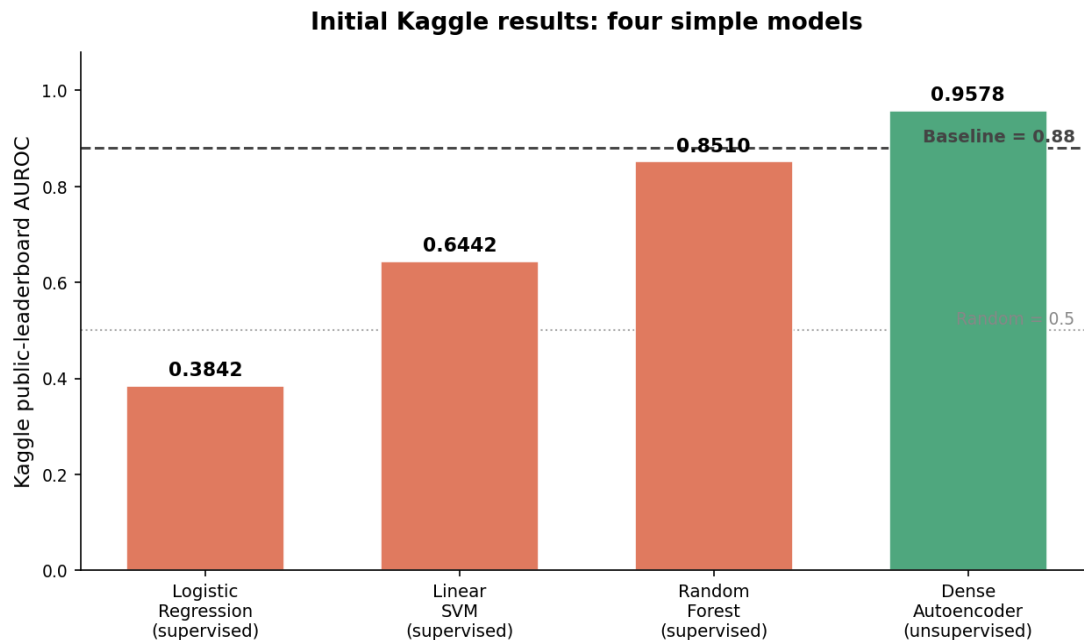


Figure 15: Recapping Simple Models AUROC

5.8 First ensemble (RF + AE)

Our first instinct after seeing the individual results was to combine the best supervised model (Random Forest, 0.85103) with the best unsupervised model (Dense Autoencoder, 0.958) using equal weights (0.5 each). The reasoning was that RF catches non-linear interaction rules that the AE might compress away in its 8-unit bottleneck, while the AE catches anomaly types that no synthetic label simulated.

The ensemble scored **0.88460**, lower than the Autoencoder alone. This result suggests that the two models were not adding enough useful diversity to each other. As discussed in class, an ensemble only improves performance when the combined models make different kinds of errors or capture complementary patterns. In this case, both models used the same 37-feature representation, and the Random Forest was still shaped by the synthetic labels used during training. Since those labels mostly simulated return-magnitude spikes, the Random Forest remained biased toward that specific anomaly pattern.

As a result, averaging the Random Forest with the Autoencoder diluted the stronger Autoencoder signal instead of improving it. The issue was not mainly a hyperparameter problem; it came from the source of supervision. Any model trained on synthetic labels is structurally limited by the type of anomaly those labels create. Since our synthetic injection did not fully represent layering or quote-stuffing behavior, we needed a different approach.

6. Advanced Anomaly Detectors

Having exhausted the unsupervised method from the course (the Autoencoder) and shown that supervised approaches hit a structural ceiling, we reviewed the academic literature on unsupervised anomaly detection in high-frequency financial markets. Two methods emerged as particularly relevant: Isolation Forest (Liu, Ting and Zhou, 2008) and statistical z-score detectors (Cao et al., 2015).

6.1 *Isolation Forest*

After reviewing the literature on unsupervised anomaly detection in high-frequency financial markets, we identified Isolation Forest as one of the most relevant methods for this project. The algorithm was introduced by Liu, Ting, and Zhou in their 2008 paper *Isolation Forest*, published at the IEEE International Conference on Data Mining. The core argument of the paper is that anomalies have two properties that make them easier to isolate: they are few and they are different. Because anomalous observations are rare, they tend to occupy sparse, low-density regions of the feature space. Because they differ from the majority, a small number of random splits is often enough to separate them from everything else. Normal observations, which sit in dense regions surrounded by similar points, usually require many more splits before they can be isolated. Liu et al. argue that this asymmetry, anomalies isolated quickly, while normal points isolate slowly, can be turned directly into an anomaly score without explicitly modelling normal behavior and without requiring labelled examples of fraud.

Isolation Forest is structurally related to the Random Forest model we saw in class. In Random Forest, each tree is trained on a bootstrap sample of the labelled data. At each node, the algorithm searches across possible features and thresholds for the split that best separates the classes, usually by minimizing impurity or MSE. The final prediction for a new observation is obtained by averaging the predictions of all trees. Random Forest is fundamentally a supervised method: it needs labels, it optimizes its splits toward a target, and its quality is measured by how well it predicts on held-out labelled data. Isolation Forest follows a different logic. It does not need labels, and it does not optimize its splits toward any target. At each node, the feature and threshold are selected randomly. There is no class label, impurity measure, or supervised objective function. What gets averaged across trees is not a prediction but a path length: the number of splits needed to isolate each point. A point that is isolated after only a few splits is likely sitting in an unusual region of the feature space. A point that requires many splits to isolate is likely located in a dense region, where normal observations are expected to be. This distinction matters for our problem. Random Forest was bounded by the synthetic injection because it was supervised: it could only learn the type of anomaly we created during training. Isolation Forest does not have this constraint. It does not assume that fraud must look like the injected examples. Instead, it measures how easy or difficult it is to separate each test row from the rest of the data, making it better suited to detecting abnormal patterns that differ from the normal distribution.

Mathematical Intuition

For one isolation tree, the algorithm works as follows. Starting from a dataset X at depth d , it randomly selects a feature q , then randomly selects a split point p between the minimum and maximum value of that feature. The data is then split into two groups:

$$\begin{aligned} X_{\text{left}} &= \{x: x_q \leq p\} \\ X_{\text{right}} &= \{x: x_q > p\} \end{aligned}$$

This process is repeated recursively until the point is isolated or theFor one isolation tree, the algorithm works as follows. The tree starts with a dataset X at depth d . If only one observation remains, or if the maximum depth has been reached, the algorithm stops and returns a leaf at depth d :

$$\text{if } |X| \leq 1 \text{ or } d \geq d_{\max} : \text{return leaf with depth } d$$

If the stopping condition is not reached, the algorithm randomly selects one feature from the available F features:

$$q \leftarrow \text{Uniform}\{1, \dots, F\}$$

It then randomly selects a split threshold between the minimum and maximum value of that feature:

$$p \leftarrow \text{Uniform}[\min(X_q), \max(X_q)]$$

Using this random threshold, the dataset is split into two subsets:

$$\begin{aligned} X_L &= \{x \in X: x_q \leq p\} \\ X_R &= \{x \in X: x_q > p\} \end{aligned}$$

The same process is then repeated recursively on both subsets:

$$\text{return Node}(q, p, \text{IsolationTree}(X_L, d + 1), \text{IsolationTree}(X_R, d + 1))$$

The key output of this process is the path length. For a test observation x , the path length $h(x)$ is the number of edges the point travels from the root node to the leaf where it is isolated. Since Isolation Forest builds many random trees, the model averages the path length across all T trees:

$$E[h(x)] = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

This average path length is what turns the random tree structure into an anomaly detector. If a point is unusual, it should be isolated quickly in many trees, giving it a small $E[h(x)]$. If a point is normal, it should be harder to isolate because it is surrounded by many similar observations, giving it a larger $E[h(x)]$.

To convert this path length into a normalized anomaly score, Isolation Forest uses the expected path length of a random binary search tree built from n samples:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

where:

$$H(k) = \ln(k) + \gamma$$

and $\gamma \approx 0.5772$ is the Euler-Mascheroni constant. The normalized anomaly score is then:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

This formula gives higher scores to observations with shorter path lengths. If $s(x, n)$ is close to 1, the point was isolated quickly and is likely anomalous. If the score is close to 0.5, the point has an average isolation depth. If the score is closer to 0, the point was difficult to isolate and is likely normal.

In our implementation, we fitted Isolation Forest on the RobustScaler-transformed normal pool. We used `n_estimators=400` so the average path length would be stable across many random trees, and `max_samples=512` so each tree would be trained on a small random subset of the normal data. This also increases the diversity of the trees, because each tree sees a slightly different part of the normal distribution. We used `contamination=0.01` as a conservative setting, and `n_jobs=-1` to parallelize the training.

In sklearn, the method `score_samples()` returns a signed score where lower values correspond to more abnormal observations. To make the direction easier to interpret and to combine with other anomaly scores later, we negated the scores and normalized them to the $[0, 1]$ range:

$$\text{score}_{IF}(i) = \frac{-\text{score_samples}(x_i) - \min_j}{\max_j - \min_j + \varepsilon}$$

where ε is added to avoid division by zero. After this transformation, a higher $\text{score}_{IF}(i)$ means that the observation is more likely to be anomalous.

Isolation Forest achieved a public AUROC of **0.95373**.

6.2 Z-score detector

The second method we found in the literature was a statistical z-score approach to anomaly detection. Unlike Isolation Forest, which builds trees and measures isolation depth, the z-score detector does not train any model at all. The idea comes from classical statistical process control, a field concerned with detecting when a manufacturing or financial process has drifted outside its normal operating range. In the context of financial market surveillance, Cao et al. (2015) applied

this kind of deviation-based scoring to limit order book data and showed that simple statistical thresholds on individual features can be surprisingly effective at flagging manipulative behaviour, especially when the manipulation leaves extreme values in the raw data. Their work demonstrated that measuring how far each observation deviates from the normal distribution of that specific instrument, rather than using a global pooled baseline, significantly improves detection reliability in multi-asset settings.

The core idea is straightforward: for each test observation, we measure how far each feature value is from what is considered normal for that stock symbol, expressed in units of standard deviation. An observation that sits many standard deviations away from its expected range is flagged as suspicious. The further it sits, the higher its anomaly score.

Because each stock has its own behavior and volatility, we do not compare all symbols using one global mean and standard deviation. Instead, we compute the statistics separately for each symbol using only the normal training rows. This follows the EDA finding from **Section 3.3**, where we saw that some variables had very different ranges across symbols. A value that is extreme for one stock may be normal for another, so a pooled baseline would make the anomaly score less reliable.

Mathematical Intuition

For a set of n normal training observations $x_1, x_2, \dots, x_n$, the mean is computed as:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

and the standard deviation is computed as:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

The z-score tells us how many standard deviations an observation is away from the mean. For one observation x_i , the z-score is:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Because anomalies can happen in both directions, meaning a value can be unusually high or unusually low, we use the absolute value of the z-score:

$$|z_i| = \frac{|x_i - \mu|}{\sigma}$$

In the code, the z-score is computed per symbol and per feature. For a test row i , feature f , and symbol s , we calculate:

$$|z_{i,f,s}| = \frac{|x_{i,f} - \mu_{f,s}|}{\sigma_{f,s}}$$

where $\mu_{f,s}$ and $\sigma_{f,s}$ are the mean and standard deviation of feature f , computed only from the normal training rows for symbol s .

After computing the z-score for each feature, we summarize the feature-level z-scores using two main statistics. The first is the maximum absolute z-score across all features:

$$z_{\max,i} = \max_f |z_{i,f,s}|$$

This captures the strongest abnormal signal in the row. The second is the average absolute z-score across all F features:

$$\bar{z}_i = \frac{1}{F} \sum_{f=1}^F |z_{i,f,s}|$$

This captures whether the row is broadly unusual across many features, instead of being extreme in only one feature.

The raw anomaly score combines these z-score signals with the burst-related features engineered in **Section 4.1**. The formula used is:

$$s_z(i) = 3.0 \cdot \log(1 + z_{\max,i}) + 1.0 \cdot \log(1 + \bar{z}_i) + 8.0 \cdot \mathbf{1}[\text{burst}_{10-49}] + 4.0 \cdot \mathbf{1}[\text{burst}_{50+}] + 1.0 \cdot \log(1 + \text{burst_size}_i)$$

Here, $z_{\max,i}$ is given a larger weight because the EDA showed that some test values reached extremely high z-scores, around $z \approx 326$, while normal training values were usually much smaller.

The average z-score $\bar{z}_i$ is included as a secondary signal because it helps identify rows that are moderately unusual across several features. The burst indicators are included because **Section 3.5** showed that burst order sizes were strongly linked to larger return movements. In particular, burst orders of size 10–49 had the strongest signal, so they received the highest burst weight.

We use logarithmic compression because raw z-scores can become extremely large. The log transformation reduces the effect of very extreme observations while still preserving the ordering of the scores. This matters because AUROC evaluates ranking quality, not the exact numerical value of each score. Since:

$$g(z) = \log(1 + z)$$

is monotone increasing, applying it keeps the ranking direction the same but prevents the score distribution from being dominated by one or two extreme rows.

Finally, the raw anomaly score is min-max normalized so that all scores fall between 0 and 1:

$$\text{score}_z(i) = \frac{s_z(i) - \min_j s_z(j)}{\max_j s_z(j) - \min_j s_z(j) + \varepsilon}$$

The small ε is added to avoid division by zero in case all raw scores are the same. The final result, $\text{score}_z(i)$, is the normalized statistical anomaly score used by this detector.

The z-score detector achieved a public AUROC of **0.99742** which is the highest score of any individual detector by a wide margin. The `burst_10_49` term carried most of the signal: the 173× return ratio identified in Section 3.5 meant that any test row with a burst size in the 10-49 range was an almost certain fraud. The z-score terms then captured additional fraud rows with extreme individual feature values that the burst features did not reach.

Despite this strong result, we did not submit the z-score alone as our final answer for two reasons. First, the public leaderboard evaluates only 50% of the test rows, a score of 0.997 on one half does not guarantee the same on the other, particularly if the private half contains manipulation patterns that are subtler or structured differently. Second, the z-score treats every feature independently. It cannot detect anomalies that arise from an unusual relationship between features rather than from any single extreme value. For example, a row where `ReturnBid1` and `ReturnAsk1` decouple from their usual joint behaviour without either being individually extreme.

6.3 Two-model ensemble (IF + AE)

After testing Isolation Forest and the Autoencoder individually, we wanted to see whether combining them would improve the score. The intuition was reasonable: Isolation Forest measures how quickly a row can be isolated in the feature space, while the Autoencoder measures how poorly a row reconstructs from the learned normal structure. These are two different ways of asking whether a row looks unusual, so we expected the two detectors to catch at least some different cases.

We combined them using a simple equal-weighted average of their normalized scores and submitted the result to Kaggle. The combined model achieved a public AUROC of **0.95383**, which was essentially the same as the Autoencoder alone at **0.95785**. In other words, the ensemble did not create a meaningful improvement. This result was still useful because it told us that Isolation Forest and the Autoencoder were largely flagging the same rows. Both models were trained on the

same 1.54 million normal observations and applied to the same 37-feature space. Although their mechanisms are different, both are sensitive to rows that deviate from the joint normal distribution: Isolation Forest flags rows in sparse regions of the feature space, while the Autoencoder flags rows that do not reconstruct well from the learned normal structure. In practice, these two ideas overlapped more than expected. This ruled out the assumption that combining two unsupervised detectors would automatically add useful diversity. The mechanism matters, but the coverage matters more. Since both models were trained on the same data, used the same features, and responded to similar types of abnormal rows, averaging them did not add much new information. To improve beyond this point, we needed a detector that looked at the problem from a different angle. This led us to think of incorporating the z score as part of the ensemble as detailed in section 6.2.

6.4 Final 3-way ensemble (Z-score + IF + AE)

In this section, we detail the use of two different weighting strategies for the ensembling of our 3 unsupervised models.

Strategy A: Grid Search

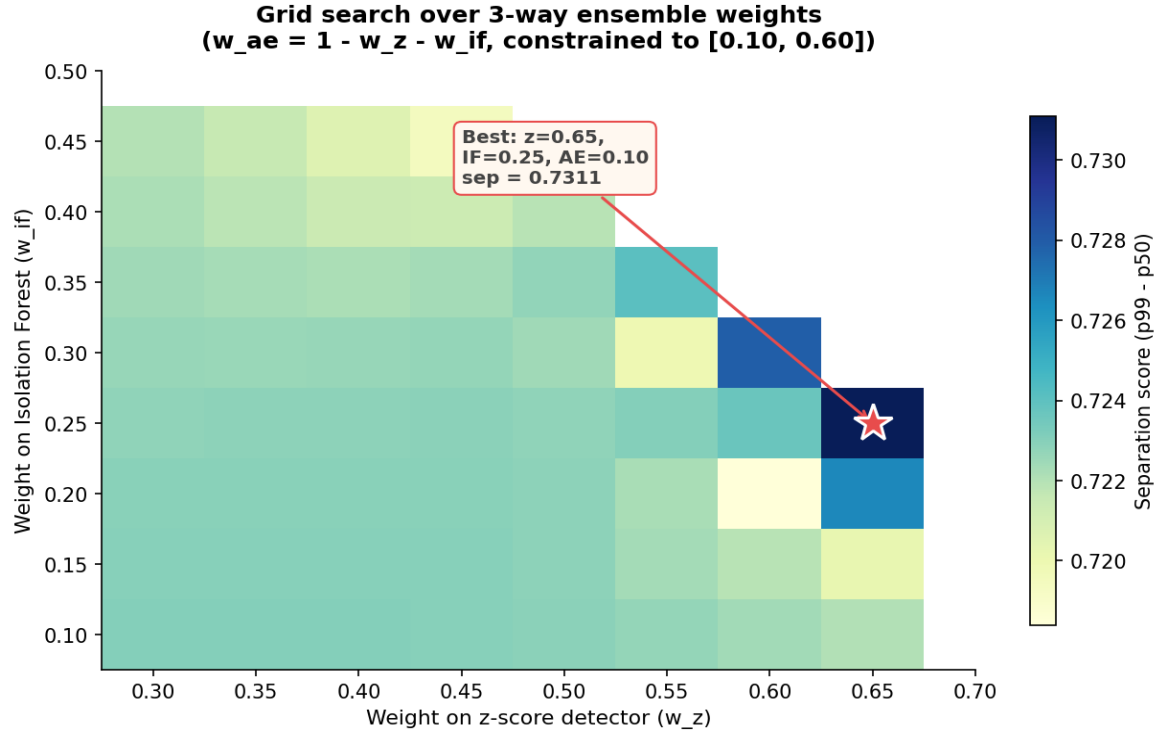
The first approach was data-driven. We ran a grid search over possible weight combinations for the three detectors in 5% increments, requiring the weights to sum to 1. Since the test labels are hidden, we could not directly optimize AUROC. Instead, we used a proxy metric: the separation between the 99th percentile and the 50th percentile of the combined score.

The idea was that a useful anomaly detector should push the most suspicious rows toward the top of the score distribution while leaving ordinary rows closer to the middle. A larger gap between the 99th and 50th percentiles suggests stronger separation between highly suspicious rows and the rest of the sample.

The best grid-search combination placed most of the weight on the z-score detector, with smaller weights on Isolation Forest and the Autoencoder:

$$\text{score}_{final} = 0.65 \cdot \text{score}_z + 0.25 \cdot \text{score}_{IF} + 0.10 \cdot \text{score}_{AE}$$

This combination achieved a public AUROC of **0.99473**.



Strategy B: EDA-Justified Weights

The second approach used the EDA findings directly instead of optimizing a proxy metric. We assigned weights based on which detector was best positioned to capture the manipulation signals identified earlier in the report.

The z-score detector received the largest weight, **0.50**, because the EDA showed that the strongest signals were exactly the ones z-score captures well: extreme individual feature values and burst-order structure. These included the $11.6\times$ range violation and the $173\times$ return ratio for burst sizes between 10 and 49.

Isolation Forest and the Autoencoder each received a weight of **0.25**. Isolation Forest contributed multivariate sparsity information, while the Autoencoder contributed reconstruction-based information. However, neither had an individual signal as dominant as the z-score detector, so they were given smaller but still meaningful weights.

The EDA-weighted formula was:

$$\text{score}_{EDA} = 0.50 \cdot \text{score}_z + 0.25 \cdot \text{score}_{IF} + 0.25 \cdot \text{score}_{AE}$$

This combination achieved a public AUROC of **0.99356**.

Comparing the two strategies, the grid-searched ensemble performed slightly better on the public leaderboard, with an AUROC of **0.99473**, compared to **0.99356** for the EDA-weighted ensemble. The difference was small, but both strategies pointed to the same conclusion: the z-score detector

carried the strongest anomaly signal, while Isolation Forest and the Autoencoder added smaller supporting signals.

For the final submission, we selected the grid-searched ensemble as the primary model. This choice was based on three reasons. First, it achieved a stronger public leaderboard score. Second, the selected weights still made sense from the EDA perspective, since the z-score detector received the majority weight and the two unsupervised detectors were kept as secondary components. Third, it offered a more balanced submission than the z-score detector alone, because it kept some multivariate and reconstruction-based coverage for anomalies that may not appear as single-feature extremes. The EDA-weighted ensemble was kept as a secondary submission because it was more directly interpretable. Its weights were easier to justify from the EDA findings, even though its public score was slightly lower.

7. Results and Discussion

7.1 Leaderboard progression

Table or bar chart showing the Kaggle leaderboard score at each milestone, from baseline through the final ensemble.

Model / Ensemble	Kaggle ROC-AUC
Baseline (provided)	0.88
Logistic Regression	0.38342
Random Forest	0.85103
SVM (linear)	0.64425
Autoencoder	0.95785
RF + AE ensemble	0.88460
IF + AE ensemble	0.95383
Final 3-way ensemble	0.99473

7.2 What worked and why

Looking back at the full leaderboard progression, the pattern is clear: every meaningful performance jump came from moving further away from supervised learning and closer to methods that learn directly from the normal data. Understanding why requires going back to the EDA findings from Part 1.

The first and most important thing that worked was the shift to unsupervised detectors. The supervised models, Logistic Regression, Random Forest, and SVM, were all trained on synthetic anomalies generated by scaling up return columns. This injection was a practical necessity since train and val contain no labelled fraud, but it came with a drawback: every supervised model learned to detect one specific kind of anomaly, the return-magnitude spike, and nothing else. The EDA in Section 3.5 identified three manipulation types, each leaving a different fingerprint in different feature families. Pump-and-dump lives in the return sequence; layering lives in the cancellation-to-trade ratio; quote stuffing lives in the burst-order structure. A model trained to recognise scaled-up returns will never reliably detect the other two patterns, regardless of how well it is tuned. This is why the supervised ceiling sat at 0.855 and could not be broken by ensemble combinations. The problem was not the models themselves, but the training signal they were given.

The Autoencoder broke through that ceiling precisely because it was not given any training signal about what fraud looks like. It only learned what normal looks like. Anything that deviates from the normal manifold, whether it is a return-magnitude spike, an unusual cancellation pattern, or a burst-order sequence, reconstructs poorly and gets a high anomaly score. The fact that it scored 0.958 without ever seeing a fraud example demonstrated that the structure of normal LOB data is distinct enough from fraud that reconstruction error alone is a reliable separating signal.

The z-score detector worked for a more specific reason that connects directly back to the EDA. Section 3 showed that the strongest anomaly signals in the data are concentrated in a small number of features: the burst structure ($173\times$ return ratio for orders with 10-49 sub-IDs) and the extreme range violations (test values reaching $11.6\times$ the training maximum on return columns). These are not subtle signals buried in joint-feature structure. They are large, direct, and measurable with simple statistics. A model sophisticated enough to learn the entire 37-dimensional manifold is not needed to catch them. What is needed is a direct measurement of deviation from the normal range, computed per symbol to account for the heterogeneity identified in Section 3.3. The z-score formula does exactly this, and the weights reflect exactly what the EDA found: *burst_10_49* gets weight 8.0 because it marks the $173\times$ zone; *z_max* gets weight 3.0 because test values reach $z = 326$ while normal values sit at $z = 1-3$. The result, 0.997 alone, confirmed that when the EDA signals are strong and the features are engineered correctly, a rule-based approach can outperform a deep model.

The final ensemble worked because of diversity. The three detectors each cover a different aspect of the anomaly space. The z-score catches rows where individual features are extreme relative to their symbol's normal range. Isolation Forest catches rows that sit in sparse regions of the 37-dimensional feature space, rows that are collectively unusual even if no single feature is individually extreme. The Autoencoder catches rows whose overall feature combination was never seen during training and cannot be reproduced by the bottleneck representation learned from 1.54 million normal rows. These three failure modes are different enough that the ensemble consistently scores higher than any individual component. The grid search confirmed this: the optimal weights

placed z-score at 0.65, Isolation Forest at 0.25, and the Autoencoder at 0.10, reflecting that the z-score signal dominated but the other two contributed genuinely independent coverage on the cases the z-score alone missed.

7.3 Limitations

The first and most significant limitation is that the synthetic anomaly injection used to train the supervised models only simulates one of the three manipulation types identified in the EDA. Scaling return columns by a factor of 10 to 50 creates artificial pump-and-dump-like rows, but it does not simulate layering, which requires high cancellation activity at low trade volume, or quote stuffing, which requires a burst of rapid sub-events at moderate return magnitudes. This means the supervised models were effectively blind to two thirds of the manipulation space from the start. The unsupervised models partially compensate for this because they do not depend on the injection, but they were also not specifically designed or validated against layering and quote stuffing examples. Detection of those two patterns relied entirely on the engineered features capturing their signatures indirectly.

The second limitation concerns the per-symbol heterogeneity identified in Section 3.3. We addressed this for the z-score detector by computing all statistics per symbol, which correctly accounts for the fact that AAPL and GOOG have very different natural volatility profiles. However, Isolation Forest and the Autoencoder were both trained on the pooled five-symbol normal data. This means both models learned a single shared representation of what normal looks like across all stocks. For features like *BidSize1*, where the standard deviation varies by a factor of 3.15 across symbols, a pooled model will inevitably be less precise, it will see the full cross-symbol variance in *BidSize1* as part of the normal range, which makes it less sensitive to within-symbol deviations. Per-symbol versions of both models would likely improve performance, at the cost of five times the training and parameter complexity.

The third limitation is temporal. The training data spans eight trading dates, June 21 of each year from 2012 to 2019. All other things being equal, this provides a reasonable sample of normal LOB behaviour across different market conditions. But the test set covers only a single day, and that day is not guaranteed to be representative of the full eight-year spread. Volatility regimes change year to year: the microstructure of a 2012 NASDAQ order book is not identical to that of a 2019 one. The per-symbol means and standard deviations that back the z-score features are averages over this eight-year window, which means they may not perfectly describe what normal looked like on the specific test date. This cross-year drift is a confounding variable we could not remove without temporal modelling or date-stratified normalisation.

8. Conclusion

We started this project with a hard constraint. Both the train and validation sets were 100% normal. The test set contained hidden anomalies. There were no positive classes to fit a supervised model against. The goal was to detect three manipulation patterns: pump-and-dump, layering, and quote stuffing.

The EDA gave us three findings that drove everything downstream. The data was pre-standardized but had heavy positive tails. The test set differed from train along several confounded axes at once: symbol mix, single trading day, and intraday shape. The three manipulation types each lived in a different feature family. These findings shaped the 37-feature matrix in Section 4.

The supervised models all hit the same ceiling. We had to fabricate fraud labels through synthetic injection. The labels only mimicked the pump-and-dump fingerprint. Logistic regression dropped below random at 0.38342. SVM landed at 0.6443. Random Forest reached 0.8510 but still fell short of the 0.88 baseline.

The unsupervised path broke through that ceiling. The Dense Autoencoder learned the normal manifold directly and reached 0.9579. Isolation Forest gave us a complementary multivariate view. The z-score detector captured the strongest EDA signals through a simple weighted formula. Combining all three in a grid-searched ensemble gave a final public-leaderboard AUROC of 0.99473, well above the baseline of 0.88.

The biggest lesson is about detector diversity. Each component caught different test rows. The ensemble was stronger than any single model. When the training distribution does not match the test distribution, supervised methods struggle. Learning the normal pattern directly is more reliable.

Future work could go in several directions. Windowed temporal features would target pump-and-dump and quote stuffing more precisely. Per-symbol modelling for BidSize1 would address the heterogeneity from Section 3.3. A gradient-boosted ensemble of all detectors could improve on linear weighting. Time-conditioned thresholds would let us flag different rates of suspicious activity through the trading day.

References

- Cao, Y., Li, Y., Coleman, S., Belatreche, A., & McGinnity, T. M. (2015). Adaptive hidden Markov model with anomaly states for price manipulation detection. *IEEE Transactions on Neural Networks and Learning Systems*, 26(2), 318–330.
- Goldstein, M., & Uchida, S. (2016). A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PLOS ONE*, 11(4), e0152173.
- Grubbs, F. E. (1969). Procedures for detecting outlying observations in samples. *Technometrics*, 11(1), 1-21.
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
- Krogh, A., & Vedelsby, J. (1995). Neural network ensembles, cross validation, and active learning. *Advances in Neural Information Processing Systems*, 7, 231-38.
- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 413–422.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- Zimek, A., Campello, R. J., & Sander, J. (2013). Subsampling for efficient and effective unsupervised outlier detection ensembles. *Proceedings of KDD*, 428–436.